Διαδικτυακή Πλατφόρμα για Οπτικοποίηση Παράλληλα Εξελισσόμενων Χρονοσειρών

Αθανάσιος Μοδιώτης

Διπλωματική Εργασία

Επιβλέπων: Απόστολος Ζάρας

Ιωάννινα, Φεβρουάριος 2026

ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY OF IOANNINA

Περίληψη

Η παρούσα διπλωματική εργασία παρουσιάζει την ανάπτυξη μιας web έκδοσης της εφαρμογής Plutarch Parallel Lives, η οποία αρχικά σχεδιάστηκε ως desktop εργαλείο για την οπτικοποίηση της εξέλιξης οντοτήτων λογισμικού στο χρόνο. Ο κύριος στόχος ήταν η μετατροπή της υπάρχουσας εφαρμογής Java σε μια προσβάσιμη διαδικτυακή πλατφόρμα διατηρώντας τη βασική της λειτουργικότητα.

Η υλοποίηση περιλάμβανε τη δημιουργία ενός REST API layer με χρήση Spring Boot, το οποίο εκθέτει 17 endpoints για φόρτωση δεδομένων, ομαδοποίηση (clustering), οπτικοποίηση, ανίχνευση μοτίβων και εξαγωγή δεδομένων. Βασικό στοιχείο αποτελεί το σύστημα διαχείρισης συνεδριών στη μνήμη (ProjectStore), το οποίο επιτρέπει πολλαπλούς ταυτόχρονους χρήστες μέσω thread-safe δομών δεδομένων (ConcurrentHashMap) και αυτόματου καθαρισμού ληγμένων συνεδριών.

Το frontend αναπτύχθηκε ως Single Page Application με χρήση vanilla JavaScript, HTML5 και CSS, παρέχοντας ένα διαδραστικό Parallel Lives Diagram με χρωματική κωδικοποίηση κελιών, παράθυρα πληροφοριών οντοτήτων και επισήμανση μοτίβων. Το σύστημα υποστηρίζει διάφορες μορφές εισόδου συμπεριλαμβανομένων αρχείων CSV/TSV και datasets Schema Evolution.

Πραγματοποιήθηκε εκτεταμένος έλεγχος με 49 αυτοματοποιημένα tests (24 integration tests για το REST API και 25 unit tests για τον ProjectStore), διασφαλίζοντας την αξιοπιστία και ορθότητα της εφαρμογής. Η αξιολόγηση έδειξε ότι η web εφαρμογή αναπαράγει επιτυχώς τη λειτουργικότητα της desktop έκδοσης προσφέροντας παράλληλα τα πλεονεκτήματα της προσβασιμότητας μέσω browser και της υποστήριξης πολλαπλών χρηστών.

Λέξεις Κλειδιά: REST API, Spring Boot, Διαχείριση Συνεδριών, Single Page Application, Εξέλιξη Λογισμικού, Parallel Lives Diagram, Web Εφαρμογή

Abstract

This diploma thesis presents the development of a web-based version of the Plutarch Parallel Lives application, originally designed as a desktop tool for visualizing the evolution of software entities over time. The main objective was to transform the existing Java application into an accessible web platform while preserving its core functionality.

The implementation involved the creation of a REST API layer using Spring Boot, which exposes 17 endpoints for data loading, clustering, visualization, pattern detection, and data export. A key component is the in-memory session management system (ProjectStore), which enables multiple concurrent users through thread-safe data structures (ConcurrentHashMap) and automatic cleanup of expired sessions.

The frontend was developed as a Single Page Application using vanilla JavaScript, HTML5, and CSS, featuring an interactive Parallel Lives Diagram with color-coded cells, entity information modals, and pattern highlighting. The system supports various input formats including CSV/TSV files and Schema Evolution datasets.

Extensive testing was performed with 49 automated tests (24 integration tests for the REST API and 25 unit tests for the ProjectStore), ensuring reliability and correctness. The evaluation demonstrated that the web application successfully replicates the desktop functionality while offering the advantages of browser-based accessibility and multi-user support.

Keywords: REST API, Spring Boot, Session Management, Single Page Application, Software Evolution, Parallel Lives Diagram, Web Application

Πίνακας Περιεχομένων

Κεφάλαιο 1. Εισαγωγή **9**

1.1 Στόχοι της εργασίας 10

1.2 Μεθοδολογία 10

1.3 Τεχνολογίες και Εργαλεία Υλοποίησης 11

Κεφάλαιο 2. Υπόβαθρο **13**

2.1 Εισαγωγή στο Plutarch's Parallel Lives 13

2.1.1 Βασικές Λειτουργίες της Desktop Εφαρμογής 14

2.2 Εννοιολογικό Μοντέλο Παράλληλης Εξέλιξης 18

2.3 Ιστορική Εξέλιξη της Εφαρμογής 18

2.3.1 Αρχική Έκδοση (Giachos, 2015) [6] 18

2.3.2 Γενικευμένη Έκδοση (Pantelidis, 2021) [7] 18

2.3.3 Αναβαθμισμένη Έκδοση (Batsilas, 2022) [8] 19

2.4 Μοντέλο Δεδομένων 19

2.4.1 Intermediate Representation 19

2.4.2 Summarized Representation 20

2.5 Ανίχνευση Μοτίβων και Clustering 21

2.6 2 Σχέση Υποβάθρου με την Παρούσα Εργασία 21

Κεφάλαιο 3. Περιπτώσεις Χρήσης **22**

3.1 Εισαγωγή 22

3.2 Φόρτωση Δεδομένων 22

3.2.1 UC01: Φόρτωση CSV/TSV 22

3.2.2 UC02: Φόρτωση Schema Evolution 23

3.2.3 UC03: Φόρτωση Project από Φάκελο 24

3.3 Ομαδοποίηση 24

3.3.1 UC04: Εφαρμογή Clustering 24

3.4 Οπτικοποίηση 25

3.4.1 UC05: Προβολή Parallel Lives Diagram 25

3.4.2 UC06: Λεπτομέρειες Οντότητας *26*
3.4.3 UC07: Λεπτομέρειες Κελιού (Interactive Cells) *27*
3.5 Μοτίβα 27
3.5.1 UC08: Εμφάνιση Μοτίβων *27*
3.5.2 UC09: Εξαγωγή Αναφοράς Μοτίβων *28*
3.6 Ταξινόμηση 29
3.6.1 UC10: Ταξινόμηση Πίνακα *29*
3.7 Aggregation 29
3.7.1 UC11: Αλλαγή Aggregation Type *29*
3.8 Εξαγωγή 30
3.8.1 UC12: Εξαγωγή Project σε Φάκελο *30*
3.8.2 UC13: Εξαγωγή Δεδομένων *30*
3.9 Βοήθεια 31
3.9.1 UC14: Προβολή Βοήθειας (Help Modal) *31*
3.10 Σύνοψη Περιπτώσεων Χρήσης 32
Κεφάλαιο 4. Σχεδίαση 33
4.1 Εισαγωγή 33
4.2 Αρχιτεκτονική Συστήματος 33
4.2.1 Διαγράμματα UML *34*
4.3 Σχεδίαση REST API 38
4.4 Διαχείριση Συνεδριών (Session Management) 39
4.4.1 Διαχείριση Σφαλμάτων (GlobalExceptionHandler) *41*
4.5 Frontend 41
4.5.1 Γενική Επισκόπηση Διεπαφής Χρήστη *42*
4.5.2 Παράθυρο Ρύθμισης Clustering (Configure Clustering) *43*
4.5.3 Παράθυρο Πληροφοριών Οντότητας (Entity Info) *44*
4.5.4 Διαδραστικά Κελιά και Λεπτομέρειες Κελιού (Cell Details) *44*
4.6 Νέα Αρχεία Υλοποίησης 47
Κεφάλαιο 5. Έλεγχος 48
5.1 Εισαγωγή 48
5.2 Έλεγχος REST API 48

5.2.1 Διαχείριση Projects (Project Management Tests) *49*
5.2.2 Φόρτωση Δεδομένων (Data Loading Tests) *49*
5.2.3 Ομαδοποίηση (Clustering Tests) *49*
5.2.4 Ανάκτηση Δεδομένων Πίνακα (Chart Data Tests) *50*
5.2.5 Ταξινόμηση και Μοτίβα (Sorting and Patterns Tests) *50*
5.2.6 Λεπτομέρειες Κελιού (Cell Details Tests) *51*
5.2.7 Εξαγωγή και Αποθήκευση (Export Tests) *51*
5.2.8 Στατιστικά Συνεδριών (Session Statistics Tests) *52*
5.3 Έλεγχος ProjectStore 53
5.3.1 Δημιουργία Project (createProject Tests) *53*
5.3.2 Ανάκτηση Project (get Tests) *53*
5.3.3 Διαγραφή Project (delete Tests) *54*
5.3.4 Μέτρηση Ενεργών Projects (getActiveProjectCount Tests) *54*
5.3.5 Καθαρισμός Συνεδριών (Session Cleanup Tests) *55*
5.3.6 Ασφάλεια Νημάτων (Thread Safety Tests) *55*
5.3.7 Ολοκληρωμένος Κύκλος Ζωής (Lifecycle Tests) *55*
5.4 Εκτέλεση Tests 56
Κεφάλαιο 6. Σύνοψη 57
6.1 Επισκόπηση 57
6.2 Συνεισφορά της Εργασίας 57
6.3 Παραδοτέα 58
6.4 Μελλοντικές Επεκτάσεις 58
6.4.1 Λειτουργικές Επεκτάσεις *58*
6.4.2 Τεχνικές Επεκτάσεις *59*
6.5 Συμπέρασμα 59
Κεφάλαιο 7. Βιβλιογραφία 60

Κεφάλαιο 1. Εισαγωγή

Η παρούσα διπλωματική εργασία έχει ως αντικείμενο τη μετατροπή της εφαρμογής desktop Plutarch Parallel Lives (PPL) σε διαδικτυακή (web) εφαρμογή, με στόχο τη βελτίωση της προσβασιμότητας, της επεκτασιμότητας και της ευχρηστίας της.

Η εφαρμογή Plutarch Parallel Lives αποτελεί ένα σύστημα οπτικοποίησης παράλληλα εξελισσόμενων χρονοσειρών και έχει σχεδιαστεί για την ανάλυση κοινοτήτων ομότιμων οντοτήτων που συν-εξελίσσονται στον χρόνο. Η βασική ιδέα της εφαρμογής βασίζεται στο Διάγραμμα Παράλληλων Ζωών (Parallel Lives Diagram – PLD), ένα δισδιάστατο μοντέλο απεικόνισης όπου οι γραμμές αντιστοιχούν σε οντότητες, οι στήλες σε διακριτές χρονικές στιγμές και τα κελιά κωδικοποιούν ποσοτικά ή ποιοτικά χαρακτηριστικά της εξέλιξης κάθε οντότητας.

Η μεθοδολογία και το θεωρητικό υπόβαθρο της εφαρμογής παρουσιάζονται στην εργασία «Parallel Lives Diagrams for Co-Evolving Communities and their Application to Schema Evolution» [6], που εκπονήθηκε από τους Φάνη Γιάχο, Νίκο Παντελίδη, Χρήστο Μπατσίλα, Απόστολο Ζάρρα και Παναγιώτη Βασιλειάδη, στο πλαίσιο ερευνητικής δραστηριότητας του Πανεπιστημίου Ιωαννίνων.

Η αρχική εφαρμογή υλοποιήθηκε ως desktop εφαρμογή, με γραφικό περιβάλλον βασισμένο σε JavaFX/Swing [9], γεγονός που απαιτούσε την τοπική εγκατάστασή της στον υπολογιστή του χρήστη. Παρότι η υλοποίηση αυτή παρείχε πλούσια λειτουργικότητα, περιόριζε τη δυνατότητα άμεσης πρόσβασης και χρήσης της εφαρμογής από διαφορετικά περιβάλλοντα και συσκευές.

Στόχος της παρούσας εργασίας είναι η δημιουργία μιας web έκδοσης της εφαρμογής PPL, η οποία θα επιτρέπει τη χρήση της μέσω ενός σύγχρονου φυλλομετρητή (browser), χωρίς την ανάγκη εγκατάστασης, διατηρώντας παράλληλα τη βασική φιλοσοφία και τις δυνατότητες ανάλυσης της αρχικής εφαρμογής.

Η μετάβαση σε web περιβάλλον προσφέρει σημαντικά πλεονεκτήματα, όπως:

- Προσβασιμότητα, καθώς οι χρήστες μπορούν να χρησιμοποιήσουν την εφαρμογή από οποιαδήποτε συσκευή διαθέτει browser.
- Ευκολία χρήσης, αφού δεν απαιτείται εγκατάσταση Java ή άλλων εξαρτήσεων.
- Απλούστερη συντήρηση, καθώς οι ενημερώσεις πραγματοποιούνται κεντρικά στον server.
- Μελλοντική επεκτασιμότητα, με δυνατότητα προσθήκης λειτουργιών συνεργατικής χρήσης (collaboration).

Η εφαρμογή PPL έχει χρησιμοποιηθεί κυρίως για τη μελέτη της εξέλιξης σχημάτων βάσεων δεδομένων, ωστόσο το υποκείμενο μοντέλο της είναι γενικεύσιμο και μπορεί να εφαρμοστεί και σε άλλα πεδία, όπως η ανάλυση λογισμικού, οικονομικών δεδομένων ή πληθυσμιακών χρονοσειρών.

1.1 Στόχοι της εργασίας

Οι βασικοί στόχοι της παρούσας διπλωματικής εργασίας είναι οι εξής:

- Η δημιουργία REST API, το οποίο εκθέτει το σύνολο των λειτουργιών του backend της εφαρμογής Plutarch Parallel Lives, επιτρέποντας την αποσύνδεση της επιχειρησιακής λογικής από το γραφικό περιβάλλον.
- Η υλοποίηση σύγχρονου web frontend, το οποίο θα παρέχει όλες τις δυνατότητες της υπάρχουσας desktop έκδοσης, αξιοποιώντας τις δυνατότητες των σύγχρονων web τεχνολογιών.
- Η υποστήριξη πολλαπλών ταυτόχρονων χρηστών, μέσω μηχανισμών διαχείρισης συνεδριών (session management), ώστε κάθε χρήστης να μπορεί να εργάζεται ανεξάρτητα με τα δικά του δεδομένα.
- Η δημιουργία και εκτέλεση δοκιμών (tests) για τον νέο κώδικα που εισάγεται, με σκοπό τη διασφάλιση της ορθής λειτουργίας και της αξιοπιστίας της web εφαρμογής.

1.2 Μεθοδολογία

Η μεθοδολογία που ακολουθήθηκε στην παρούσα εργασία βασίζεται στην αρχή της ελάχιστης τροποποίησης του υπάρχοντος backend κώδικα. Ο βασικός στόχος ήταν η διατήρηση όσο το δυνατόν περισσότερου από τον υπάρχοντα κώδικα, περιορίζοντας τις αλλαγές στο απολύτως απαραίτητο. Η κύρια προσέγγιση ήταν η δημιουργία ενός νέου επιπέδου REST API, το οποίο «τυλίγει» τον υπάρχοντα κώδικα και εκθέτει τη λειτουργικότητά του ως web υπηρεσία. Όπου απαιτήθηκαν επεκτάσεις στον

υπάρχοντα κώδικα, όπως στην κλάση MainController για την υποστήριξη νέων λειτουργιών (π.χ. ανάκτηση λεπτομερειών κελιού), αυτές υλοποιήθηκαν με τρόπο που δεν επηρεάζει την υπάρχουσα λειτουργικότητα και διατηρεί τη συμβατότητα με την desktop έκδοση.

Η επιλογή αυτή κρίθηκε ιδιαίτερα κατάλληλη, καθώς η αρχική εφαρμογή ήταν ήδη υλοποιημένη με ξεκάθαρο διαχωρισμό επιπέδων, γεγονός που επέτρεψε την επαναχρησιμοποίηση της επιχειρησιακής λογικής χωρίς επεμβάσεις. Η αρχιτεκτονική αυτή διευκόλυνε τη μετάβαση στο web περιβάλλον και μείωσε σημαντικά την πολυπλοκότητα της υλοποίησης.

Η συγκεκριμένη προσέγγιση παρουσιάζει σημαντικά πλεονεκτήματα:

- Μειώνει τον κίνδυνο σφαλμάτων, καθώς ο υπάρχων και δοκιμασμένος κώδικας παραμένει αμετάβλητος.
- Διασφαλίζει τη συντηρησιμότητα, αφού οι αλλαγές περιορίζονται στο REST API και στο web frontend.
- Επιτρέπει την επαναχρησιμοποίηση του backend, τόσο από τη web έκδοση όσο και από ενδεχόμενες μελλοντικές εφαρμογές.
- Ενισχύει την επεκτασιμότητα, καθώς το API μπορεί να αποτελέσει βάση για νέους clients ή πρόσθετες λειτουργίες.

Η προσέγγιση αυτή ακολουθεί σύγχρονες αρχές ανάπτυξης λογισμικού [1], σύμφωνα με τις οποίες η αποσύνδεση backend και frontend θεωρείται κρίσιμη για τη δημιουργία ευέλικτων και ανθεκτικών συστημάτων.

1.3 Τεχνολογίες και Εργαλεία Υλοποίησης

Για την υλοποίηση της web έκδοσης της εφαρμογής Plutarch Parallel Lives χρησιμοποιήθηκε ένα σύνολο σύγχρονων τεχνολογιών και εργαλείων ανάπτυξης λογισμικού, τα οποία επιλέχθηκαν με γνώμονα τη σταθερότητα, την επεκτασιμότητα και τη συμβατότητα με τον υπάρχοντα κώδικα της desktop έκδοσης.

Η εφαρμογή αναπτύχθηκε στη γλώσσα προγραμματισμού Java [9], με στόχο την πλατφόρμα Java 11, διατηρώντας τη συμβατότητα με τον πυρήνα της αρχικής εφαρμογής. Για την υλοποίηση του backend και του REST API επιλέχθηκε το πλαίσιο ανάπτυξης Spring Boot [2] (έκδοση 2.7.18), το οποίο παρέχει ενσωματωμένη υποστήριξη για web εφαρμογές, RESTful υπηρεσίες και μηχανισμούς διαχείρισης αιτημάτων HTTP.

Παρότι η εφαρμογή μετατράπηκε σε web περιβάλλον, διατηρήθηκαν εξαρτήσεις από τη βιβλιοθήκη JavaFX Base [9] (έκδοση 11), καθώς μέρος της επιχειρησιακής λογικής του

συστήματος βασίζεται σε δομές δεδομένων όπως οι ObservableList, οι οποίες χρησιμοποιούνται στον υπάρχοντα πυρήνα της εφαρμογής.

Για τη διαδικασία ελέγχου και δοκιμών του συστήματος χρησιμοποιήθηκε το πλαίσιο JUnit 5 [3] (Jupiter), σε συνδυασμό με το Spring Boot Starter Test, επιτρέποντας τη συγγραφή και εκτέλεση unit και integration tests για το REST API και τις βασικές λειτουργίες του backend.

Η διαχείριση των εξαρτήσεων και η διαδικασία build της εφαρμογής πραγματοποιήθηκαν μέσω του Apache Maven [10], με χρήση των plugins maven-compiler-plugin για τη μεταγλώττιση του κώδικα και spring-boot-maven-plugin για τη δημιουργία εκτελέσιμου αρχείου JAR της εφαρμογής. Η εκτέλεση των δοκιμών κατά το build υλοποιήθηκε μέσω του plugin maven-surefire-plugin.

Τέλος, η ανάπτυξη και η διαχείριση του έργου πραγματοποιήθηκαν στο περιβάλλον Eclipse IDE, το οποίο χρησιμοποιήθηκε για τη συγγραφή του κώδικα, την εκτέλεση της εφαρμογής και τη διαχείριση του Maven project.

Κεφάλαιο 2. Υπόβαθρο

2.1 Εισαγωγή στο Plutarch's Parallel Lives

Το Plutarch's Parallel Lives (PPL) [6,7,8] είναι ένα εργαλείο οπτικοποίησης που έχει σχεδιαστεί για τη μελέτη παράλληλα εξελισσόμενων χρονοσειρών, δηλαδή συνόλων δεδομένων που αποτελούνται από ομότιμες οντότητες οι οποίες εξελίσσονται ταυτόχρονα μέσα σε ένα κοινό χρονικό πλαίσιο.

Η ονομασία του εργαλείου προέρχεται από το έργο Βίοι Παράλληλοι του αρχαίου Έλληνα ιστορικού Πλουτάρχου, στο οποίο παρουσιάζονται συγκριτικά οι ζωές σημαντικών προσωπικοτήτων. Αντίστοιχα, στο PPL κάθε οντότητα διαθέτει μια «ζωή» η οποία παρακολουθείται και αναλύεται σε συνάρτηση με τις υπόλοιπες οντότητες της ίδιας κοινότητας.

Η βασική μορφή οπτικοποίησης της εφαρμογής είναι το Parallel Lives Diagram (PLD), το οποίο αναπαριστά τα δεδομένα σε δισδιάστατη μορφή, με:

- τις οντότητες να αντιστοιχούν στις γραμμές,
- τις χρονικές στιγμές (beats) να αντιστοιχούν στις στήλες,
- και τα κελιά να κωδικοποιούν τη δραστηριότητα ή την κατάσταση κάθε οντότητας σε κάθε χρονική στιγμή.

Η εφαρμογή υποστηρίζει ποικίλα σύνολα δεδομένων, όπως:

- εξέλιξη σχημάτων βάσεων δεδομένων (schema evolution),
- πληθυσμιακά δεδομένα χωρών (π.χ. GapMinder),
- χρηματιστηριακές χρονοσειρές,
- καθώς και κάθε τύπο δεδομένων που μπορεί να μοντελοποιηθεί ως σύνολο ομότιμων οντοτήτων με κοινό χρονικό άξονα.

2.1.1 Βασικές Λειτουργίες της Desktop Εφαρμογής

Η desktop έκδοση της εφαρμογής PPL παρέχει ένα πλήρες σύνολο λειτουργιών για τη φόρτωση, επεξεργασία, ανάλυση, οπτικοποίηση και εξαγωγή δεδομένων παράλληλης εξέλιξης. Οι βασικές λειτουργίες περιγράφονται αναλυτικά στη συνέχεια.

Φόρτωση Δεδομένων (Data Loading)

Η εφαρμογή υποστηρίζει πολλαπλούς τρόπους φόρτωσης δεδομένων, προσαρμοσμένους σε διαφορετικές πηγές και μορφές:

Αρχεία TSV/CSV: Η πιο απλή μορφή εισόδου είναι ένα μεμονωμένο αρχείο με διαχωρισμένες τιμές. Το αρχείο πρέπει να ακολουθεί συγκεκριμένη δομή με στήλες για το όνομα οντότητας, τη χρονική στιγμή, και τις τιμές μετρήσεων. Η εφαρμογή αναγνωρίζει αυτόματα τον τύπο διαχωριστικού και αναλύει τα δεδομένα.

Φάκελοι Schema Evolution: Για την ανάλυση εξέλιξης σχημάτων βάσεων δεδομένων, η εφαρμογή δέχεται φακέλους που περιέχουν τα εξειδικευμένα αρχεία εξόδου του εργαλείου Schema Evolution Analysis: το αρχείο SchemaHeartbeat.tsv (που περιέχει τις χρονικές στιγμές και τις γενικές μετρήσεις), το αρχείο tables_DetailedStats.tsv (με αναλυτικές πληροφορίες ανά πίνακα), και το αρχείο transitions.csv (με τις μεταβάσεις μεταξύ εκδόσεων). Η εφαρμογή συνδυάζει αυτά τα αρχεία για να δημιουργήσει πλήρη εικόνα της εξέλιξης.

Φόρτωση Αποθηκευμένου Project: Η εφαρμογή επιτρέπει τη φόρτωση προηγουμένως αποθηκευμένων projects μέσω φακέλου που περιέχει τα αρχεία tem.tsv (Intermediate Representation) και προαιρετικά gpm.tsv (Summarized Representation αν έχει εκτελεστεί clustering). Αυτό επιτρέπει τη συνέχιση της εργασίας χωρίς επανυπολογισμό.

Ομαδοποίηση Δεδομένων (Clustering)

Η λειτουργία clustering είναι κεντρική για την αποτελεσματική οπτικοποίηση μεγάλων συνόλων δεδομένων. Η εφαρμογή παρέχει δύο τύπους ομαδοποίησης:

Time Clustering (Ομαδοποίηση Χρονικών Στιγμών): Χρησιμοποιεί τον αλγόριθμο Agglomerative Clustering [6] για την ομαδοποίηση διαδοχικών beats σε phases. Ο χρήστης καθορίζει τον επιθυμητό αριθμό φάσεων και τα βάρη απόστασης (distance weights) που ρυθμίζουν τη σχετική σημασία μεταξύ χρονικής εγγύτητας και ομοιότητας αλλαγών. Η παράμετρος changesWeight κυμαίνεται από 0.0 (ομαδοποίηση αποκλειστικά βάσει χρόνου) έως 1.0 (ομαδοποίηση αποκλειστικά βάσει αλλαγών).

Entity Clustering (Ομαδοποίηση Οντοτήτων): Ομαδοποιεί οντότητες με παρόμοιο κύκλο ζωής σε EntityGroups. Τα κριτήρια ομαδοποίησης περιλαμβάνουν τη χρονική στιγμή γέννησης (birth beat), τη χρονική στιγμή θανάτου (death beat), και τη συνολική διάρκεια ζωής. Οντότητες που «γεννήθηκαν» και «πέθαναν» σε παρόμοιες χρονικές στιγμές τοποθετούνται στην ίδια ομάδα.

Μετά την εκτέλεση του clustering, η εφαρμογή δημιουργεί τη Συνοπτική Αναπαράσταση (Summarized Representation) η οποία αποτελείται από GroupPhaseMeasurements (GPMs) που συνοψίζουν τα δεδομένα σε επίπεδο ομάδων και φάσεων.

Οπτικοποίηση (Visualization)

Η κεντρική λειτουργία οπτικοποίησης είναι το Parallel Lives Diagram (PLD), ένας δισδιάστατος πίνακας με χρωματική κωδικοποίηση:

- **Χρωματική Κλίμακα Μετρήσεων:** Τα κελιά χρωματίζονται ανάλογα με την τιμή της επιλεγμένης μέτρησης, χρησιμοποιώντας κλίμακα που κυμαίνεται από ανοιχτές αποχρώσεις (χαμηλές τιμές) σε έντονες (υψηλές τιμές). Η κλίμακα μπορεί να είναι γραμμική ή λογαριθμική ανάλογα με το εύρος των τιμών.
- **Κωδικοποίηση Κατάστασης Ζωής:** Ειδικοί χρωματισμοί υποδεικνύουν τα γεγονότα ζωής: μπλε για birth (γέννηση οντότητας), μαύρο για death (θάνατος οντότητας), πράσινες αποχρώσεις για active (εν ζωή οντότητα, με μετρήσεις), και λευκό/γκρι για inactive (πριν τη γέννα/μετά τον θάνατο).
- **Πρώτη Στήλη Κατάστασης:** Η πρώτη στήλη του πίνακα εμφανίζει συνοπτικά την τρέχουσα κατάσταση κάθε οντότητας: ανοιχτό μπλε αν η οντότητα είναι ακόμα «ζωντανή» στο τέλος της παρατήρησης, ανοιχτό κόκκινο αν έχει «πεθάνει».
- **Επιλογή Aggregation Type:** Ο χρήστης μπορεί να επιλέξει ποιες κατηγορίες αλλαγών θα συμπεριληφθούν στην οπτικοποίηση: μόνο εισαγωγές (SUM_OF_INSERTIONS), μόνο διαγραφές (SUM_OF_DELETIONS), μόνο ενημερώσεις (SUM_OF_UPDATES), ή συνδυασμούς αυτών (π.χ. SUM_OF_ALL για όλες τις αλλαγές). Αυτό είναι ιδιαίτερα χρήσιμο για δεδομένα Schema Evolution όπου κάθε αλλαγή κατηγοριοποιείται ως insertion, deletion, ή update.

Ταξινόμηση Οντοτήτων (Entity Sorting)

Η εφαρμογή παρέχει πολλαπλές επιλογές ταξινόμησης των γραμμών του πίνακα για τη διευκόλυνση της ανάλυσης:

- **BIRTH_ASCENDING / BIRTH_DESCENDING:** Ταξινόμηση βάσει χρονικής στιγμής γέννησης (αύξουσα ή φθίνουσα). Αυτό επιτρέπει την εύκολη αναγνώριση οντοτήτων που εμφανίστηκαν νωρίς ή αργά στην ιστορία.
- **ACTIVITY_ASCENDING / ACTIVITY_DESCENDING:** Ταξινόμηση βάσει συνολικής δραστηριότητας (αριθμός ή ένταση αλλαγών). Επιτρέπει την αναγνώριση των πιο ή λιγότερο ενεργών οντοτήτων.
- **LIFE_DURATION_ASCENDING / LIFE_DURATION_DESCENDING:** Ταξινόμηση βάσει διάρκειας ζωής (αριθμός beats που η οντότητα ήταν ενεργή). Επιτρέπει τη σύγκριση μακροβιότητας μεταξύ οντοτήτων.

Ανίχνευση και Διαχείριση Μοτίβων (Pattern Detection)

Μία από τις πιο σημαντικές λειτουργίες της εφαρμογής είναι η αυτόματη ανίχνευση χαρακτηριστικών μοτίβων στα δεδομένα. Τα μοτίβα αυτά αποκαλύπτουν σημαντικά γεγονότα στην εξέλιξη της κοινότητας:

- **Multiple Births (Πολλαπλές Γεννήσεις):** Ανιχνεύεται όταν πολλές οντότητες «γεννιούνται» ταυτόχρονα ή σε σύντομο χρονικό διάστημα. Στο πλαίσιο της εξέλιξης σχημάτων βάσεων δεδομένων, αυτό μπορεί να υποδηλώνει μεγάλη αναδιοργάνωση ή εισαγωγή νέου module. Επισημαίνεται με μπλε χρώμα.
- **Multiple Deaths (Πολλαπλοί Θάνατοι):** Ανιχνεύεται όταν πολλές οντότητες «πεθαίνουν» ταυτόχρονα. Μπορεί να υποδηλώνει αφαίρεση deprecated λειτουργικότητας ή μεγάλο refactoring. Επισημαίνεται με κόκκινο χρώμα.
- **Multiple Updates (Πολλαπλές Ενημερώσεις):** Ανιχνεύεται όταν πολλές οντότητες παρουσιάζουν υψηλή δραστηριότητα ταυτόχρονα. Υποδηλώνει περίοδο έντονων αλλαγών στο σύστημα. Επισημαίνεται με πορτοκαλί χρώμα.
- **Ladder Pattern (Μοτίβο Σκάλας):** Ανιχνεύεται όταν οι οντότητες ακολουθούν διαδοχικό κύκλο ζωής, δηλαδή κάθε μία γεννιέται λίγο μετά την προηγούμενη, δημιουργώντας ένα διαγώνιο μοτίβο στον πίνακα. Επισημαίνεται με μωβ χρώμα.
- Τα ανιχνευμένα μοτίβα επισημαίνονται οπτικά στον πίνακα με ειδικούς δείκτες (markers) και μπορούν να εξαχθούν σε αναφορά για περαιτέρω ανάλυση.

Εξαγωγή και Αποθήκευση (Export & Save)

Η εφαρμογή παρέχει πολλαπλές επιλογές για την εξαγωγή και αποθήκευση δεδομένων και αποτελεσμάτων:

- **Export Project (Εξαγωγή Project):** Δημιουργεί έναν φάκελο (ή ZIP αρχείο) που περιέχει όλα τα απαραίτητα αρχεία για τη μελλοντική φόρτωση του project: το αρχείο tem.tsv με την Ενδιάμεση Αναπαράσταση, το αρχείο gpm.tsv με τη Συνοπτική Αναπαράσταση (αν έχει εκτελεστεί clustering), και μεταδεδομένα όπως οι παράμετροι clustering. Αυτό επιτρέπει τη διατήρηση και διαμοιρασμό αναλύσεων.
- **Save Data (Αποθήκευση Δεδομένων):** Αποθηκεύει τα τρέχοντα δεδομένα σε μορφή TSV για χρήση σε άλλες εφαρμογές ή για αρχειοθέτηση.
- **Pattern Report (Αναφορά Μοτίβων):** Εξάγει αναφορά με τα ανιχνευμένα μοτίβα, περιλαμβάνοντας τον τύπο κάθε μοτίβου, τις εμπλεκόμενες οντότητες, τις χρονικές στιγμές εμφάνισης, και στατιστικά στοιχεία όπως η συχνότητα και η ένταση.
- **Diagram Export (Εξαγωγή Διαγράμματος):** Η desktop έκδοση επιτρέπει την εξαγωγή του οπτικοποιημένου διαγράμματος σε μορφή εικόνας (PNG, SVG) για χρήση σε παρουσιάσεις ή δημοσιεύσεις.

Πληροφορίες Οντοτήτων (Entity Information)

Η εφαρμογή παρέχει δυνατότητα προβολής αναλυτικών πληροφοριών για κάθε οντότητα ή ομάδα οντοτήτων:

- **Entity Details:** Προβολή αναλυτικών στοιχείων για μεμονωμένη οντότητα, συμπεριλαμβανομένων: ονόματος, χρονικής στιγμής γέννησης και θανάτου, συνολικής διάρκειας ζωής, συνολικής δραστηριότητας (άθροισμα μετρήσεων), και τυχόν μοτίβων στα οποία συμμετέχει.
- **EntityGroup Details:** Μετά το clustering, προβολή πληροφοριών για ομάδα οντοτήτων: λίστα μελών της ομάδας, κοινά χαρακτηριστικά (εύρος birth/death), και συγκεντρωτικά στατιστικά.
- **Phase Details:** Προβολή πληροφοριών για φάση: εύρος beats που περιλαμβάνει, συνολικός αριθμός αλλαγών, αριθμός births και deaths εντός της φάσης.
- Οι πληροφορίες αυτές εμφανίζονται μέσω αναδυόμενων παραθύρων (dialogs) κατά την επιλογή του αντίστοιχου στοιχείου στη διεπαφή.

2.2 Εννοιολογικό Μοντέλο Παράλληλης Εξέλιξης

Η θεωρητική βάση του PPL στηρίζεται στη μοντελοποίηση κοινοτήτων ομότιμων οντοτήτων που συν-εξελίσσονται στον χρόνο. Για να είναι ένα σύνολο δεδομένων συμβατό με το μοντέλο, απαιτούνται τα εξής:

- Ένα διακριτό χρονικό σύστημα (timeline) αποτελούμενο από χρονικές στιγμές (beats).
- Ένα σύνολο οντοτήτων (entities) που αποτελούν μια κοινότητα.
- Η δυνατότητα οι οντότητες να εμφανίζονται (birth) ή να εξαφανίζονται (death) κατά τη διάρκεια του χρόνου.
- Ένα ή περισσότερα μετρήσιμα μεγέθη (measurement types) που αποτυπώνουν τη δραστηριότητα των οντοτήτων.
- Ο πυρήνας του μοντέλου είναι ο Parallel Lives Matrix (PLM), ένας δισδιάστατος πίνακας διαστάσεων Entities × TimeBeats, όπου κάθε κελί περιέχει ένα αντικείμενο TimeEntityMeasurement (TEM), το οποίο συνδέει μια οντότητα με μια χρονική στιγμή και τις αντίστοιχες μετρήσεις.

Η αναπαράσταση αυτή επιτρέπει όχι μόνο την οπτικοποίηση, αλλά και την αυτοματοποιημένη ανάλυση της εξέλιξης μιας κοινότητας, μέσω εντοπισμού χαρακτηριστικών μοτίβων.

2.3 Ιστορική Εξέλιξη της Εφαρμογής

Η εφαρμογή PPL αναπτύχθηκε σταδιακά μέσα από τρεις διπλωματικές εργασίες:

2.3.1 Αρχική Έκδοση (Giachos, 2015) [6]

Η πρώτη έκδοση επικεντρώθηκε στη schema evolution και εισήγαγε το PLD ως εργαλείο οπτικοποίησης. Παρουσιάστηκαν δύο αλγόριθμοι ιεραρχικής (agglomerative) ομαδοποίησης:

ένας για τη συγχώνευση χρονικών στιγμών σε φάσεις,

και ένας για την ομαδοποίηση οντοτήτων με παρόμοια εξελικτική συμπεριφορά.

2.3.2 Γενικευμένη Έκδοση (Pantelidis, 2021) [7]

Η δεύτερη φάση εισήγαγε το Intermediate Representation, ένα γενικευμένο μοντέλο δεδομένων που επιτρέπει τη μετατροπή διαφορετικών μορφών χρονοσειρών σε κοινή αναπαράσταση. Με αυτόν τον τρόπο, η εφαρμογή έπαψε να περιορίζεται αποκλειστικά σε δεδομένα βάσεων δεδομένων.

2.3.3 Αναβαθμισμένη Έκδοση (Batsilas, 2022) [8]

Η τρίτη έκδοση επικεντρώθηκε στη βελτίωση της απόδοσης και της ανάλυσης, με:

μετάβαση σε Swing JTable [8] για καλύτερη διαχείριση μεγάλων δεδομένων,

και προσθήκη αλγορίθμων ανίχνευσης μοτίβων [8], όπως μαζικές γεννήσεις, θάνατοι και σταδιακές επεκτάσεις.

2.4 Μοντέλο Δεδομένων

2.4.1 Intermediate Representation

Η Ενδιάμεση Αναπαράσταση (Intermediate Representation) [7] αποτελεί τη βάση του μοντέλου δεδομένων της εφαρμογής. Σκοπός της είναι να μετατρέψει ετερογενή δεδομένα από διάφορες πηγές (αρχεία CSV/TSV, δεδομένα εξέλιξης σχημάτων βάσεων δεδομένων, κ.λπ.) σε μια ενιαία δομή που επιτρέπει την ομοιόμορφη επεξεργασία και οπτικοποίηση. Το Intermediate Representation αποτελείται από τις εξής βασικές οντότητες:

Beat (Χρονική Στιγμή): Κάθε beat αντιπροσωπεύει μια διακριτή χρονική στιγμή στην εξέλιξη του συστήματος. Στο πλαίσιο της εξέλιξης σχημάτων βάσεων δεδομένων, ένα beat αντιστοιχεί σε μια έκδοση (version) του σχήματος. Σε άλλες εφαρμογές, θα μπορούσε να αντιστοιχεί σε ημέρα, μήνα, ή οποιαδήποτε άλλη χρονική μονάδα. Κάθε beat έχει μοναδικό αναγνωριστικό, όνομα (συνήθως η ημερομηνία ή ο αριθμός έκδοσης), και χρονοσφραγίδα για τη διάταξη.

Entity (Οντότητα): Κάθε entity αντιπροσωπεύει ένα μέλος της «κοινότητας» που παρακολουθείται. Στην περίπτωση των βάσεων δεδομένων, οι οντότητες είναι οι πίνακες (tables) του σχήματος. Κάθε οντότητα έχει κύκλο ζωής: γεννιέται (birth) όταν εμφανίζεται για πρώτη φορά, είναι ενεργή (active) όσο υπάρχει και αλλάζει, και πεθαίνει (death) όταν αφαιρεθεί από το σύστημα. Αυτός ο κύκλος ζωής είναι θεμελιώδης για την κατανόηση της παράλληλης εξέλιξης.

Measurement Type (Τύπος Μέτρησης): Κάθε measurement type ορίζει ένα μετρήσιμο χαρακτηριστικό της οντότητας. Για πίνακες βάσεων δεδομένων, τυπικές μετρήσεις περιλαμβάνουν: τον αριθμό στηλών (columns), τον αριθμό εγγραφών (rows), ή τον αριθμό αλλαγών (changes) μεταξύ διαδοχικών εκδόσεων. Η εφαρμογή υποστηρίζει πολλαπλούς τύπους μετρήσεων ταυτόχρονα, επιτρέποντας πολυδιάστατη ανάλυση.

TimeEntityMeasurement (TEM): Το TEM είναι η θεμελιώδης μονάδα δεδομένων που συνδυάζει τρεις διαστάσεις: μια συγκεκριμένη οντότητα (entity), μια συγκεκριμένη χρονική στιγμή (beat), και τις αντίστοιχες μετρήσεις (measurements). Κάθε TEM περιέχει επίσης πληροφορία για την κατάσταση της οντότητας εκείνη τη στιγμή: αν γεννήθηκε, αν πέθανε, αν ήταν ενεργή, ή αν ήταν ανενεργή (δηλαδή υπήρχε αλλά δεν είχε αλλαγές). Ο Parallel Lives Matrix (PLM) δημιουργείται ως ένας πίνακας με σειρές τις οντότητες και στήλες τα beats, όπου κάθε κελί περιέχει ένα TEM.

2.4.2 Summarized Representation

Η Συνοπτική Αναπαράσταση (Summarized Representation) είναι απαραίτητη για την υποστήριξη μεγάλων συνόλων δεδομένων. Όταν ο αριθμός των beats ή των entities είναι υπερβολικά μεγάλος για αποτελεσματική οπτικοποίηση, η εφαρμογή χρησιμοποιεί αλγορίθμους ομαδοποίησης (clustering) για να δημιουργήσει μια συμπυκνωμένη αναπαράσταση που διατηρεί τα ουσιώδη χαρακτηριστικά των δεδομένων. Η συνοπτική αναπαράσταση αποτελείται από:

- **Phase (Φάση):** Μια phase είναι μια ομάδα διαδοχικών beats που παρουσιάζουν παρόμοια χαρακτηριστικά. Ο αλγόριθμος Agglomerative Clustering χρησιμοποιείται για την ομαδοποίηση, λαμβάνοντας υπόψη τόσο τη χρονική εγγύτητα όσο και την ομοιότητα στον αριθμό αλλαγών. Κάθε phase έχει ένα εύρος beats (π.χ. beats 5-12) και συνοψίζει τις μετρήσεις όλων των beats που περιέχει μέσω συναρτήσεων συνάθροισης (sum, average, max, min, κ.λπ.).
- **EntityGroup (Ομάδα Οντοτήτων):** Μια EntityGroup είναι μια ομάδα οντοτήτων με παρόμοιο κύκλο ζωής. Ο αλγόριθμος ομαδοποίησης εξετάζει τη χρονική στιγμή γέννησης (birth beat), τη χρονική στιγμή θανάτου (death beat), και τη συνολική διάρκεια ζωής κάθε οντότητας. Οντότητες που γεννήθηκαν και πέθαναν σε παρόμοιες χρονικές στιγμές ομαδοποιούνται μαζί, επιτρέποντας την αναγνώριση μοτίβων συμπεριφοράς.
- **GroupPhaseMeasurement (GPM):** Το GPM είναι η συνοπτική εκδοχή του TEM. Κάθε GPM αντιπροσωπεύει τη συνάθροιση όλων των TEMs που ανήκουν σε μια συγκεκριμένη ομάδα οντοτήτων (EntityGroup) και μια συγκεκριμένη φάση (Phase). Το GPM περιέχει τη συναθροισμένη τιμή των μετρήσεων, την κατάσταση της ομάδας στη φάση (αν περιέχει births, deaths, ή ενεργές οντότητες), και πληροφορίες για πιθανά μοτίβα (patterns) που ανιχνεύτηκαν. Αυτά τα GPMs είναι αυτά που τελικά απεικονίζονται ως κελιά στο Parallel Lives Diagram της web εφαρμογής.

2.5 Ανίχνευση Μοτίβων και Clustering

Η εφαρμογή υποστηρίζει ανίχνευση μοτίβων όπως:

- Multiple Births,
- Multiple Deaths,
- Multiple Updates,
- Ladder patterns.

Τα μοτίβα αυτά επιτρέπουν τον εντοπισμό σημαντικών γεγονότων στην εξέλιξη μιας κοινότητας, όπως μαζικές αλλαγές ή σταδιακές μεταβάσεις.

2.6 2 Σχέση Υποβάθρου με την Παρούσα Εργασία

Η ξεκάθαρη αρθρωτή αρχιτεκτονική και το γενικευμένο μοντέλο δεδομένων του PPL [6,7,8] καθιστούν δυνατή τη μετάβαση της εφαρμογής σε web περιβάλλον. Ο σαφής διαχωρισμός μεταξύ επιχειρησιακής λογικής και παρουσίασης αποτέλεσε βασικό παράγοντα για την υλοποίηση REST API, το οποίο αναλύεται στα επόμενα κεφάλαια.

Κεφάλαιο 3. Περιπτώσεις Χρήσης

3.1 Εισαγωγή

Στο παρόν κεφάλαιο παρουσιάζονται οι περιπτώσεις χρήσης (use cases) της web εφαρμογής Plutarch Parallel Lives. Οι περιπτώσεις χρήσης περιγράφουν τις βασικές λειτουργίες του συστήματος από την οπτική γωνία του χρήστη και αποτυπώνουν τον τρόπο με τον οποίο αυτός αλληλεπιδρά με την εφαρμογή για τη φόρτωση, ανάλυση, οπτικοποίηση και εξαγωγή δεδομένων.

Η παρουσίαση οργανώνεται θεματικά, ακολουθώντας τη φυσική ροή χρήσης της εφαρμογής.

3.2 Φόρτωση Δεδομένων

3.2.1 UC01: Φόρτωση CSV/TSV

UC	UC01
Title	Φόρτωση CSV/TSV
Actors	Τελικός χρήστης
Preconditions	Η εφαρμογή έχει ξεκινήσει χωρίς προβλήματα.
Description	Η περίπτωση χρήσης UC01 αφορά τη φόρτωση αρχείων χρονοσειρών σε μορφή CSV ή TSV. Ο χρήστης επιλέγει ένα αρχείο που περιέχει δεδομένα παράλληλα εξελισσόμενων οντοτήτων με κοινό χρονικό σύστημα και το σύστημα μετατρέπει τα δεδομένα στο Intermediate Representation.

Basic Flow	1. Ο χρήστης πατάει το κουμπί "Open File" στη γραμμή εργαλείων και επιλέγει αρχείο CSV/TSV από το σύστημα αρχείων. 2. Το σύστημα αναλύει τη δομή του αρχείου. 3. Τα δεδομένα μετατρέπονται στο Intermediate Representation. 4. Το σύστημα εμφανίζει επιβεβαίωση επιτυχούς φόρτωσης.
Postconditions	Τα δεδομένα έχουν φορτωθεί επιτυχώς σε μορφή Intermediate Representation.
Alternative Flow	Σε περίπτωση σφάλματος ανάλυσης του αρχείου, το σύστημα εμφανίζει κατάλληλο μήνυμα σφάλματος.

3.2.2 UC02: Φόρτωση Schema Evolution

UC	UC02
Title	Φόρτωση Schema Evolution
Actors	Τελικός χρήστης
Preconditions	Η εφαρμογή έχει ξεκινήσει χωρίς προβλήματα.
Description	Η περίπτωση χρήσης UC02 αφορά τη φόρτωση δεδομένων εξέλιξης σχημάτων βάσεων δεδομένων. Ο χρήστης επιλέγει φάκελο που περιέχει τα απαραίτητα αρχεία του dataset και το σύστημα αναλύει τα δεδομένα, εξάγοντας τις μετρήσεις και δημιουργώντας την αντίστοιχη ενδιάμεση αναπαράσταση.
Basic Flow	1. Ο χρήστης πατάει το κουμπί "Open Folder" στη γραμμή εργαλείων και επιλέγει φάκελο με δεδομένα Schema Evolution. 2. Το σύστημα εντοπίζει και αναλύει τα απαραίτητα αρχεία του dataset. 3. Εξάγονται οι μετρήσεις και δημιουργείται η ενδιάμεση αναπαράσταση. 4. Το σύστημα εμφανίζει επιβεβαίωση επιτυχούς φόρτωσης.
Postconditions	Τα δεδομένα Schema Evolution έχουν φορτωθεί επιτυχώς.

Alternative Flow	Σε περίπτωση ελλιπών αρχείων, το σύστημα ενημερώνει τον χρήστη.

3.2.3 UC03: Φόρτωση Project από Φάκελο

UC	UC03
Title	Φόρτωση Project από Φάκελο
Actors	Τελικός χρήστης
Preconditions	Η εφαρμογή έχει ξεκινήσει χωρίς προβλήματα.
Description	Η περίπτωση χρήσης UC03 επιτρέπει τη φόρτωση ενός αποθηκευμένου project μέσω φακέλου. Ο φάκελος περιέχει αρχεία ενδιάμεσης ή/και συνοπτικής αναπαράστασης (π.χ. tem.tsv, gpm.tsv), επιτρέποντας στον χρήστη να συνεχίσει προηγούμενη εργασία χωρίς επαναφόρτωση ή επανυπολογισμό.
Basic Flow	1. Ο χρήστης πατάει το κουμπί "Open Folder" στη γραμμή εργαλείων και επιλέγει φάκελο που περιέχει αποθηκευμένο project. 2. Το σύστημα εντοπίζει τα αρχεία tem.tsv ή/και gpm.tsv. 3. Τα αρχεία φορτώνονται άμεσα χωρίς επανυπολογισμό. 4. Το σύστημα αποκαθιστά την κατάσταση του project.
Postconditions	Το project έχει αποκατασταθεί στην προηγούμενη κατάσταση.
Alternative Flow	Εάν ο φάκελος δεν περιέχει τα απαιτούμενα αρχεία, εμφανίζεται μήνυμα σφάλματος.

3.3 Ομαδοποίηση

3.3.1 UC04: Εφαρμογή Clustering

UC	UC04
Title	Εφαρμογή Clustering
Actors	Τελικός χρήστης

Preconditions	Δεδομένα έχουν φορτωθεί επιτυχώς (UC01/UC02/UC03).
Description	Η περίπτωση χρήσης UC04 αφορά την εφαρμογή αλγορίθμων clustering για τη συμπίεση του πίνακα δεδομένων. Το σύστημα ομαδοποιεί χρονικές στιγμές και οντότητες, μειώνοντας τη διάσταση του πίνακα. Ο χρήστης ρυθμίζει Time Clustering, Entity Clustering και Distance Weights μέσω του παραθύρου Configure Clustering.
Basic Flow	1. Ο χρήστης ανοίγει το παράθυρο Configure Clustering. 2. Ρυθμίζει τον αριθμό φάσεων (Time Clustering) και ομάδων οντοτήτων (Entity Clustering). 3. Προαιρετικά ρυθμίζει τα Distance Weights. 4. Το σύστημα εκτελεί τον αλγόριθμο Agglomerative Clustering. 5. Δημιουργείται η Summarized Representation (GPM). 6. Το Parallel Lives Diagram ενημερώνεται με τις νέες διαστάσεις.
Postconditions	Η Summarized Representation έχει δημιουργηθεί και το PLD εμφανίζεται.
Alternative Flow	Σε περίπτωση αδυναμίας clustering, το σύστημα ενημερώνει τον χρήστη.

3.4 Οπτικοποίηση

3.4.1 UC05: Προβολή Parallel Lives Diagram

UC	UC05
Title	Προβολή Parallel Lives Diagram
Actors	Τελικός χρήστης
Preconditions	Δεδομένα έχουν φορτωθεί και το clustering έχει εκτελεστεί.
Description	Η περίπτωση χρήσης UC05 αφορά την προβολή του Parallel Lives Diagram (PLD). Τα δεδομένα εμφανίζονται σε μορφή πίνακα με χρωματική κωδικοποίηση, η οποία αποτυπώνει την κατάσταση ή την ένταση των μετρήσεων σε κάθε κελί.

Basic Flow	1. Το σύστημα αναλαμβάνει τη λήψη δεδομένων από το endpoint GET /api/projects/{id}/chartData. 2. Δημιουργείται δυναμικά ο HTML πίνακας με χρωματική κωδικοποίηση. 3. Τα κελιά με births, deaths ή patterns επισημαίνονται με ειδικούς δείκτες. 4. Ο χρήστης βλέπει το PLD στην κεντρική περιοχή της εφαρμογής.
Postconditions	Το Parallel Lives Diagram εμφανίζεται πλήρως.
Alternative Flow	Σε περίπτωση αποτυχίας φόρτωσης, εμφανίζεται μήνυμα σφάλματος.

3.4.2 UC06: Λεπτομέρειες Οντότητας

UC	UC06
Title	Λεπτομέρειες Οντότητας
Actors	Τελικός χρήστης
Preconditions	Το PLD είναι ορατό με φορτωμένα δεδομένα.
Description	Η περίπτωση χρήσης UC06 επιτρέπει στον χρήστη να επιλέξει μια γραμμή του πίνακα και να προβάλει αναλυτικές πληροφορίες για τη συγκεκριμένη οντότητα, όπως τη διάρκεια ζωής και τη συνολική της δραστηριότητα.
Basic Flow	1. Ο χρήστης κάνει κλικ στο όνομα οντότητας ή ομάδας στον πίνακα. 2. Εμφανίζεται αναδυόμενο παράθυρο (Entity Info). 3. Παρουσιάζονται ονόματα οντοτήτων, birth/death beats και στατιστικά. 4. Ο χρήστης κλείνει το παράθυρο.
Postconditions	Ο χρήστης έχει λάβει αναλυτικές πληροφορίες για την οντότητα.
Alternative Flow	-

3.4.3 UC07: Λεπτομέρειες Κελιού (Interactive Cells)

UC	UC07
Title	Λεπτομέρειες Κελιού (Interactive Cells)
Actors	Τελικός χρήστης
Preconditions	Έχουν φορτωθεί δεδομένα και έχει εκτελεστεί clustering. Το Parallel Lives Diagram είναι ορατό στην οθόνη.
Description	Η περίπτωση χρήσης UC07 αποτελεί νέα λειτουργικότητα που υλοποιήθηκε στο πλαίσιο της παρούσας διπλωματικής εργασίας. Επιτρέπει στον χρήστη να κάνει κλικ σε οποιοδήποτε κελί του PLD και να προβάλει αναλυτικές πληροφορίες μέσω tooltip χωρίς επιπλέον αίτημα στον server.
Basic Flow	1. Ο χρήστης κάνει κλικ σε ένα κελί του πίνακα. 2. Το frontend αξιοποιεί τα δεδομένα από το GET /api/projects/{id}/chartData. 3. Εμφανίζεται tooltip με: ονόματα οντοτήτων, εύρος φάσης, τιμή, κατάσταση, μοτίβο, lifespan, entity breakdown. 4. Ο χρήστης κλείνει το tooltip κάνοντας κλικ εκτός ή πατώντας κλείσιμο.
Postconditions	Ο χρήστης έχει λάβει αναλυτικές πληροφορίες για το κελί.
Alternative Flow	-

3.5 Μοτίβα

3.5.1 UC08: Εμφάνιση Μοτίβων

UC	UC08
Title	Εμφάνιση Μοτίβων
Actors	Τελικός χρήστης

Preconditions	Το PLD εμφανίζεται με φορτωμένα δεδομένα.
Description	Η περίπτωση χρήσης UC08 αφορά την ανίχνευση και εμφάνιση μοτίβων στο PLD. Τα κελιά που συμμετέχουν σε αναγνωρισμένα μοτίβα (Multiple Births, Deaths, Updates, Ladder) επισημαίνονται με ειδικό χρωματισμό.
Basic Flow	1. Ο χρήστης ενεργοποιεί την εμφάνιση μοτίβων. 2. Το σύστημα ανακτά μοτίβα από το endpoint GET /api/projects/{id}/patterns. 3. Τα κελιά με μοτίβα επισημαίνονται χρωματικά στον πίνακα.
Postconditions	Τα μοτίβα εμφανίζονται στο PLD.
Alternative Flow	Εάν δεν ανιχνευθούν μοτίβα, ενημερώνεται ο χρήστης.

3.5.2 UC09: Εξαγωγή Αναφοράς Μοτίβων

UC	UC09
Title	Εξαγωγή Αναφοράς Μοτίβων
Actors	Τελικός χρήστης
Preconditions	Μοτίβα έχουν ανιχνευθεί (UC08).
Description	Η περίπτωση χρήσης UC09 επιτρέπει την εξαγωγή αναφοράς που περιλαμβάνει τα ανιχνευμένα μοτίβα και τα αντίστοιχα στατιστικά στοιχεία.
Basic Flow	1. Ο χρήστης επιλέγει εξαγωγή αναφοράς μοτίβων. 2. Το σύστημα δημιουργεί αναφορά με τα ανιχνευμένα μοτίβα. 3. Το αρχείο αναφοράς λαμβάνεται από τον χρήστη.
Postconditions	Η αναφορά μοτίβων έχει εξαχθεί επιτυχώς.
Alternative Flow	-

3.6 Ταξινόμηση

3.6.1 UC10: Ταξινόμηση Πίνακα

UC	UC10
Title	Ταξινόμηση Πίνακα
Actors	Τελικός χρήστης
Preconditions	Το PLD εμφανίζεται με φορτωμένα δεδομένα.
Description	Η περίπτωση χρήσης UC10 αφορά την αλλαγή της σειράς των γραμμών του πίνακα βάσει επιλεγμένων κριτηρίων, όπως χρονοθυρίδα γέννησης, διάρκεια ζωής ή συνολική δραστηριότητα.
Basic Flow	1. Ο χρήστης επιλέγει κριτήριο ταξινόμησης. 2. Το σύστημα αποστέλλει αίτημα POST /api/projects/{id}/sort. 3. Ο πίνακας ανανεώνεται με τη νέα σειρά γραμμών.
Postconditions	Ο πίνακας έχει ταξινομηθεί σύμφωνα με το επιλεγμένο κριτήριο.
Alternative Flow	-

3.7 Aggregation

3.7.1 UC11: Αλλαγή Aggregation Type

UC	UC11
Title	Αλλαγή Aggregation Type
Actors	Τελικός χρήστης
Preconditions	Το PLD εμφανίζεται με φορτωμένα δεδομένα.
Description	Η περίπτωση χρήσης UC11 επιτρέπει την επιλογή διαφορετικού τύπου συνάθροισης (aggregation type), επηρεάζοντας τον τρόπο υπολογισμού και απεικόνισης των μετρήσεων.

Basic Flow	1. Ο χρήστης επιλέγει νέο aggregation type (π.χ. SUM_OF_UPDATES, SUM_OF_INSERTIONS, SUM_OF_UPDATES). 2. Το σύστημα εφαρμόζει τον νέο τύπο συνάθροισης. 3. Ο πίνακας ανανεώνεται με τις νέες τιμές.
Postconditions	Ο πίνακας εμφανίζεται με τον νέο aggregation type.
Alternative Flow	-

3.8 Εξαγωγή

3.8.1 UC12: Εξαγωγή Project σε Φάκελο

UC	UC12
Title	Εξαγωγή Project σε Φάκελο
Actors	Τελικός χρήστης
Preconditions	Δεδομένα έχουν φορτωθεί.
Description	Η περίπτωση χρήσης UC12 αφορά την αποθήκευση της τρέχουσας κατάστασης της εφαρμογής σε φάκελο, ο οποίος περιλαμβάνει τα απαραίτητα αρχεία δεδομένων και μεταδεδομένων για μελλοντική επαναφόρτωση.
Basic Flow	1. Ο χρήστης επιλέγει εξαγωγή project. 2. Το σύστημα δημιουργεί ZIP αρχείο με τα αρχεία δεδομένων. 3. Ο χρήστης λαμβάνει το αρχείο.
Postconditions	Το project έχει αποθηκευτεί επιτυχώς.
Alternative Flow	Σε περίπτωση σφάλματος I/O, εμφανίζεται μήνυμα σφάλματος.

3.8.2 UC13: Εξαγωγή Δεδομένων

UC	UC13
Title	Εξαγωγή Δεδομένων

Actors	Τελικός χρήστης
Preconditions	Δεδομένα έχουν φορτωθεί.
Description	Η περίπτωση χρήσης UC13 επιτρέπει την εξαγωγή των δεδομένων της εφαρμογής σε αρχεία TSV.
Basic Flow	1. Ο χρήστης επιλέγει εξαγωγή δεδομένων. 2. Το σύστημα δημιουργεί αρχεία TSV. 3. Ο χρήστης λαμβάνει τα αρχεία.
Postconditions	Τα δεδομένα έχουν εξαχθεί σε μορφή TSV.
Alternative Flow	-

3.9 Βοήθεια

3.9.1 UC14: Προβολή Βοήθειας (Help Modal)

UC	UC14
Title	Προβολή Βοήθειας (Help Modal)
Actors	Τελικός χρήστης
Preconditions	Καμία - η βοήθεια είναι διαθέσιμη ανά πάσα στιγμή.
Description	Η περίπτωση χρήσης UC14 αφορά την εμφάνιση του παραθύρου βοήθειας που παρέχει στον χρήστη πληροφορίες για τη χρήση της εφαρμογής, οργανωμένες σε πέντε θεματικές ενότητες (Supported Input Formats, Quick Start, Diagram Colors, Patterns, Keyboard Shortcuts).
Basic Flow	1. Ο χρήστης πατάει το κουμπί Help ή το πλήκτρο '?'. 2. Εμφανίζεται το Help Modal με πέντε θεματικές ενότητες. 3. Ο χρήστης διαβάζει τις πληροφορίες. 4. Ο χρήστης κλείνει το παράθυρο (κουμπί κλεισίματος, κλικ εκτός, ή Escape).

Postconditions	Ο χρήστης λαμβάνει πληροφορίες για την αποτελεσματική χρήση της εφαρμογής.
Alternative Flow	-

3.10 Σύνοψη Περιπτώσεων Χρήσης

Κωδικός	**Περίπτωση Χρήσης**	**Κατηγορία**
UC01	Φόρτωση CSV/TSV	Φόρτωση
UC02	Φόρτωση Schema Evolution	Φόρτωση
UC03	Φόρτωση Project από Φάκελο	Φόρτωση
UC04	Εφαρμογή Clustering	Ομαδοποίηση
UC05	Προβολή PLD	Οπτικοποίηση
UC06	Λεπτομέρειες Οντότητας	Οπτικοποίηση
UC07	Λεπτομέρειες Κελιού (Interactive Cells)	Οπτικοποίηση
UC08	Εμφάνιση Μοτίβων	Μοτίβα
UC09	Εξαγωγή Αναφοράς	Μοτίβα
UC10	Ταξινόμηση	Ταξινόμηση
UC11	Aggregation Type	Μέτρηση
UC12	Εξαγωγή Project (Φάκελος)	Εξαγωγή
UC13	Εξαγωγή Δεδομένων	Εξαγωγή
UC14	Προβολή Βοήθειας	Βοήθεια

Κεφάλαιο 4. Σχεδίαση

4.1 Εισαγωγή

Στο παρόν κεφάλαιο παρουσιάζεται η σχεδίαση και η αρχιτεκτονική της web έκδοσης της εφαρμογής Plutarch Parallel Lives. Η βασική σχεδιαστική επιλογή της παρούσας εργασίας ήταν η ελαχιστοποίηση των τροποποιήσεων στον υπάρχοντα backend κώδικα, διατηρώντας τη σταθερότητα και τη συμβατότητα με τον πυρήνα της αρχικής εφαρμογής. Όπου κρίθηκε απαραίτητο, πραγματοποιήθηκαν στοχευμένες επεκτάσεις στις διεπαφές IMainController και MainController για την υποστήριξη νέων λειτουργιών της web έκδοσης, χωρίς ωστόσο να διαταραχθεί η υπάρχουσα λειτουργικότητα.

Η web υλοποίηση βασίστηκε στην προσθήκη νέων επιπέδων πάνω από τον υπάρχοντα κώδικα, ακολουθώντας μια πολυεπίπεδη αρχιτεκτονική που διαχωρίζει καθαρά την παρουσίαση, τη διαχείριση αιτημάτων και την επιχειρησιακή λογική.

4.2 Αρχιτεκτονική Συστήματος

Η συνολική αρχιτεκτονική της εφαρμογής αποτελείται από τρία βασικά επίπεδα, τα οποία συνεργάζονται μεταξύ τους χωρίς άμεση σύζευξη:

Επίπεδο	**Περιγραφή**
Frontend	Web εφαρμογή τύπου Single Page Application (SPA) με HTML, CSS και JavaScript
REST API	Επίπεδο διασύνδεσης υλοποιημένο με Spring Boot (DaintinessRestAPI.java)
Session Management	Διαχείριση κατάστασης χρηστών μέσω της κλάσης ProjectStore

Backend	Υπάρχουσα εφαρμογή PPL μέσω του IMainController

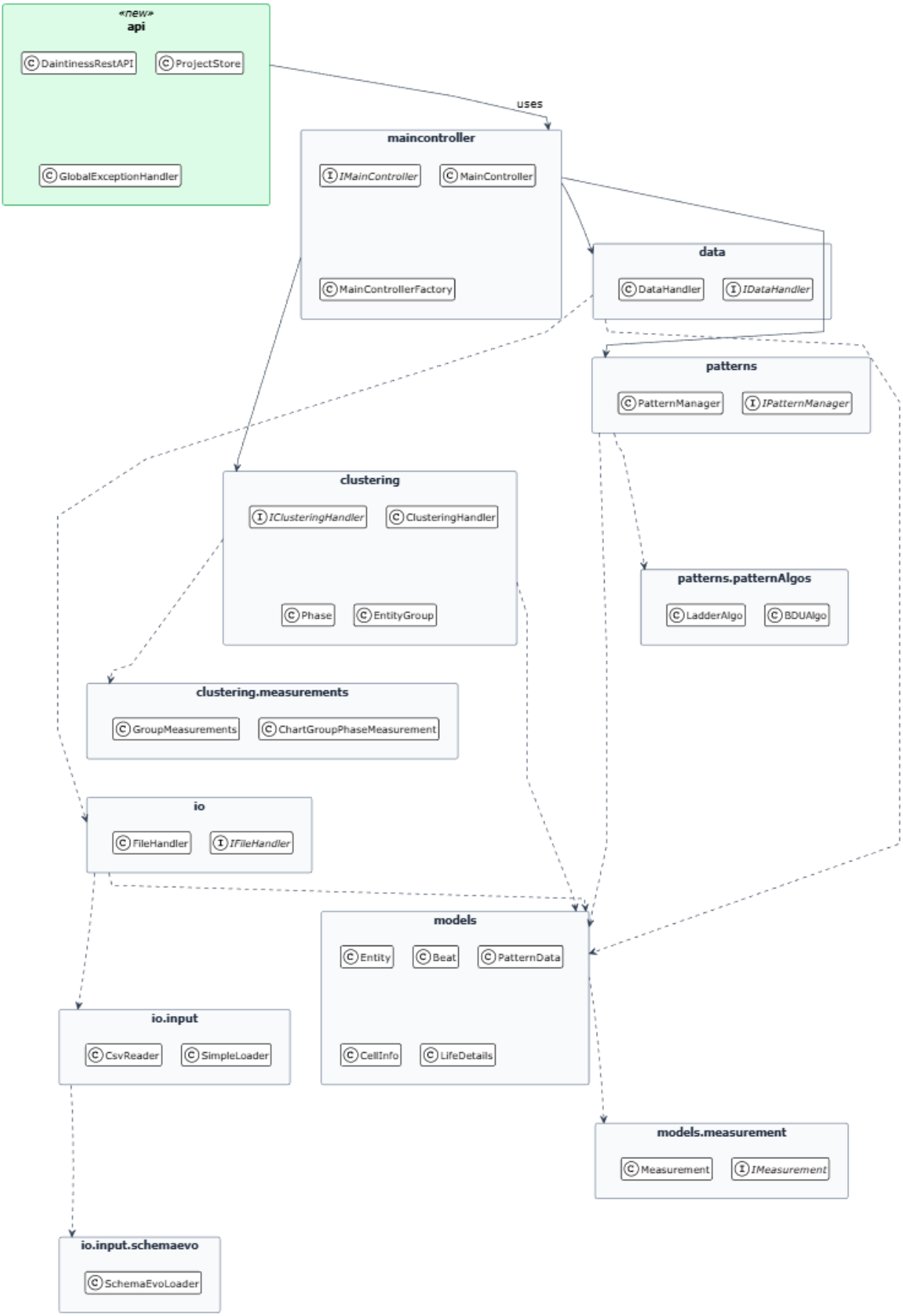
Το frontend επικοινωνεί αποκλειστικά με το REST API μέσω HTTP αιτημάτων. Το REST API λειτουργεί ως ενδιάμεσο επίπεδο που μεταφράζει τα αιτήματα του χρήστη σε κλήσεις προς τον υπάρχοντα backend κώδικα, ο οποίος παραμένει ανεξάρτητος από το web περιβάλλον.

4.2.1 Διαγράμματα UML

Για την καλύτερη κατανόηση της αρχιτεκτονικής του συστήματος, παρατίθενται τα ακόλουθα διαγράμματα UML που απεικονίζουν τη δομή και τις αλληλεπιδράσεις μεταξύ των βασικών components.

Συνολικό Διάγραμμα Αρχιτεκτονικής (Component Diagram)

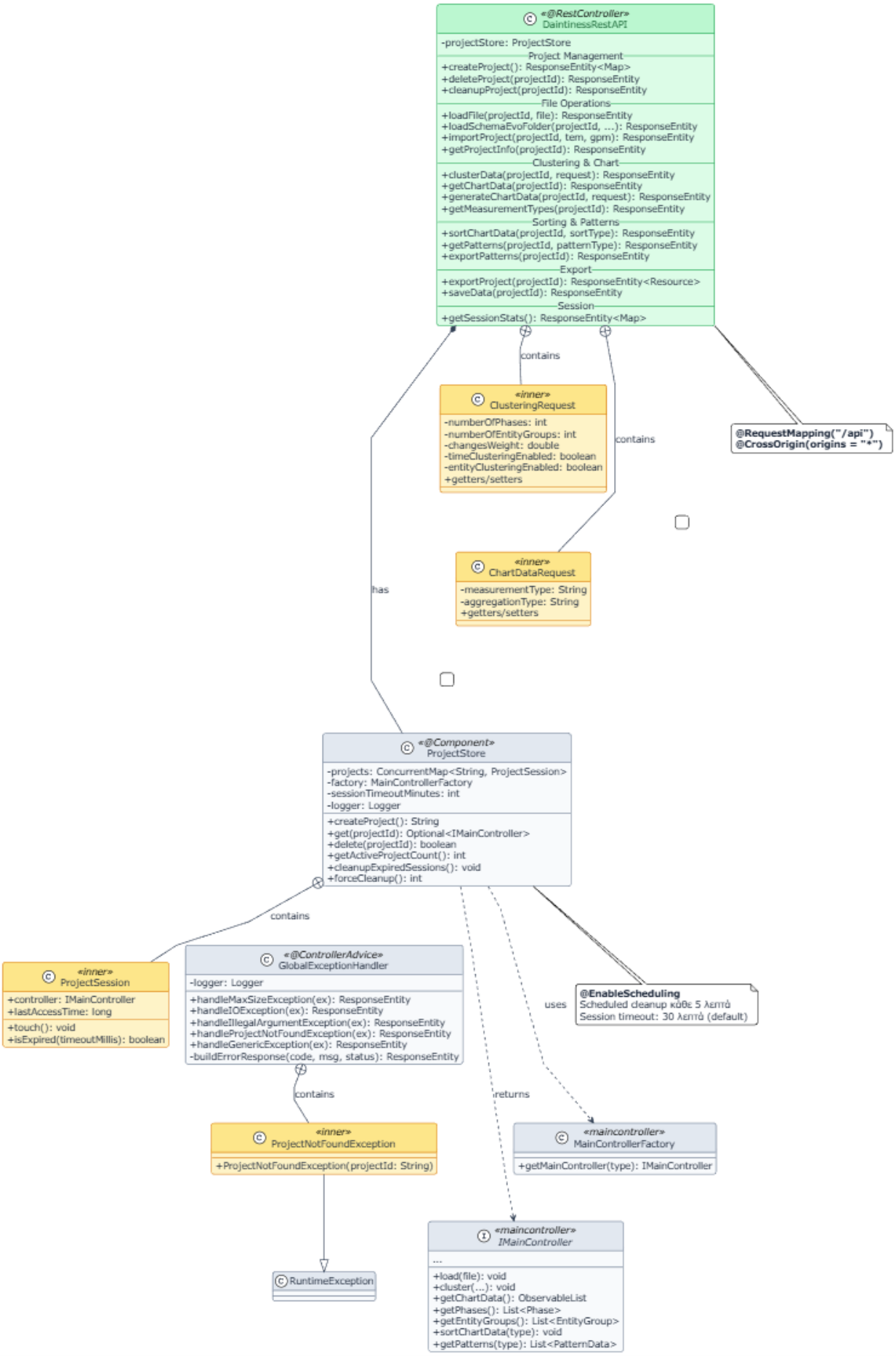
Το συνολικό διάγραμμα αρχιτεκτονικής παρουσιάζει την τριεπίπεδη δομή του συστήματος: το Frontend (Single Page Application) που εκτελείται στον browser του χρήστη, το REST API Layer που διαχειρίζεται τα HTTP αιτήματα και τη διαχείριση συνεδριών μέσω του ProjectStore, και το Backend Core που περιλαμβάνει τον υπάρχοντα κώδικα της εφαρμογής Daintiness (MainController, αλγόριθμοι clustering, pattern detection). Οι κατευθυνόμενες ακμές δείχνουν τη ροή δεδομένων και τις εξαρτήσεις μεταξύ των components.


«new»
api
DaintinessRestAPI
ProjectStore
GlobalExceptionHandler
uses
maincontroller
IMainController
MainController
MainControllerFactory
data
DataHandler
IDataHandler
patterns
PatternManager
IPatternManager
clustering
IClusteringHandler
ClusteringHandler
Phase
EntityGroup
patterns.patternAlgos
LadderAlgo
BDUAlgo
clustering.measurements
GroupMeasurements
ChartGroupPhaseMeasurement
io
FileHandler
IFileHandler
models
Entity
Beat
PatternData
CellInfo
LifeDetails
io.input
CsvReader
SimpleLoader
models.measurement
Measurement
IMeasurement
io.input.schemaevo
SchemaEvoLoader

Εικόνα 1: Συνολικό Διάγραμμα Αρχιτεκτονικής (Component Diagram)

Διάγραμμα REST API Layer (Class Diagram)

Το διάγραμμα κλάσεων του REST API Layer παρουσιάζει τη δομή του επιπέδου που υλοποιήθηκε στην παρούσα εργασία. Περιλαμβάνει την κλάση DaintinessRestController που διαχειρίζεται όλα τα HTTP endpoints, την κλάση ProjectStore που υλοποιεί τη διαχείριση συνεδριών με thread-safe αποθήκευση (ConcurrentHashMap), και τις βοηθητικές κλάσεις για την αναπαράσταση των request/response objects (DTOs). Επίσης φαίνεται η διεπαφή IMainController που ορίζει το συμβόλαιο επικοινωνίας με τον υπάρχοντα backend κώδικα.


«@RestController»
DaintinessRestAPI
-projectStore: ProjectStore
Project Management
+createProject(): ResponseEntity<Map>
+deleteProject(projectId): ResponseEntity
+cleanupProject(projectId): ResponseEntity
File Operations
+loadFile(projectId, file): ResponseEntity
+loadSchemaEvoFolder(projectId, ...): ResponseEntity
+importProject(projectId, tem, gpm): ResponseEntity
+getProjectInfo(projectId): ResponseEntity
Clustering & Chart
+clusterData(projectId, request): ResponseEntity
+getChartData(projectId): ResponseEntity
+generateChartData(projectId, request): ResponseEntity
+getMeasurementTypes(projectId): ResponseEntity
Sorting & Patterns
+sortChartData(projectId, sortType): ResponseEntity
+getPatterns(projectId, patternType): ResponseEntity
+exportPatterns(projectId): ResponseEntity
Export
+exportProject(projectId): ResponseEntity<Resource>
+saveData(projectId): ResponseEntity
Session
+getSessionStats(): ResponseEntity<Map>
contains
«inner»
ClusteringRequest
-numberOfPhases: int
-numberOfEntityGroups: int
-changesWeight: double
-timeClusteringEnabled: boolean
-entityClusteringEnabled: boolean
+getters/setters
contains
@RequestMapping("/api")
@CrossOrigin(origins = "*")
«inner»
ChartDataRequest
-measurementType: String
-aggregationType: String
+getters/setters
has
«@Component»
ProjectStore
-projects: ConcurrentMap<String, ProjectSession>
-factory: MainControllerFactory
-sessionTimeoutMinutes: int
-logger: Logger
+createProject(): String
+get(projectId): Optional<IMainController>
+delete(projectId): boolean
+getActiveProjectCount(): int
+cleanupExpiredSessions(): void
+forceCleanup(): int
contains
«inner»
ProjectSession
+controller: IMainController
+lastAccessTime: long
+touch(): void
+isExpired(timeoutMillis): boolean
«@ControllerAdvice»
GlobalExceptionHandler
-logger: Logger
+handleMaxSizeException(ex): ResponseEntity
+handleIOException(ex): ResponseEntity
+handleIllegalArgumentException(ex): ResponseEntity
+handleProjectNotFoundException(ex): ResponseEntity
+handleGenericException(ex): ResponseEntity
-buildErrorResponse(code, msg, status): ResponseEntity
uses
@EnableScheduling
Scheduled cleanup κάθε 5 λεπτά
Session timeout: 30 λεπτά (default)
contains
returns
«inner»
ProjectNotFoundException
+ProjectNotFoundException(projectId: String)
«maincontroller»
MainControllerFactory
+getMainController(type): IMainController
RuntimeException
«maincontroller»
IMainController
...
+load(file): void
+cluster(...): void
+getChartData(): ObservableList
+getPhases(): List<Phase>
+getEntityGroups(): List<EntityGroup>
+sortChartData(type): void
+getPatterns(type): List<PatternData>

Εικόνα 2: Διάγραμμα REST API Layer (Class Diagram)

4.3 Σχεδίαση REST API

Το REST API αποτελεί το κεντρικό σημείο επικοινωνίας μεταξύ frontend και backend. Υλοποιήθηκε με χρήση του πλαισίου Spring Boot [2] και εκθέτει ένα σύνολο από endpoints που αντιστοιχούν άμεσα στις περιπτώσεις χρήσης της εφαρμογής. Η σχεδίαση ακολουθεί τις αρχές του RESTful σχεδιασμού [1], χρησιμοποιώντας τα κατάλληλα HTTP methods (GET, POST, DELETE) ανάλογα με τη σημασιολογία κάθε λειτουργίας.

Η αρχιτεκτονική του REST API βασίζεται στα ακόλουθα βασικά χαρακτηριστικά:

- **Stateless Communication:** Κάθε αίτημα περιέχει όλες τις απαραίτητες πληροφορίες για την επεξεργασία του [1]. Η κατάσταση του project διαχειρίζεται στον server μέσω του ProjectStore, με κάθε project να ταυτοποιείται από μοναδικό UUID.
- **JSON Data Format:** Όλες οι απαντήσεις επιστρέφονται σε μορφή JSON [4], εξασφαλίζοντας διαλειτουργικότητα και ευκολία επεξεργασίας από το JavaScript frontend.
- **Multipart File Upload:** Για τη φόρτωση αρχείων δεδομένων (CSV/TSV), χρησιμοποιείται το πρότυπο multipart/form-data, επιτρέποντας αποτελεσματική μεταφορά μεγάλων αρχείων.
- **Consistent Error Handling:** Τα σφάλματα επιστρέφονται με κατάλληλους HTTP status codes (400 για Bad Request, 404 για Not Found, κ.λπ.) και περιγραφικά μηνύματα σε JSON μορφή.
- **Binary File Downloads:** Για τις λειτουργίες εξαγωγής, το API υποστηρίζει επιστροφή binary δεδομένων (ZIP, TSV) με τα κατάλληλα Content-Disposition headers για αυτόματη λήψη από τον browser.

Ο παρακάτω πίνακας παρουσιάζει τα βασικά endpoints του συστήματος:

Method	Endpoint	Περιγραφή
POST	/api/projects	Δημιουργία νέου project
POST	/api/projects/{id}/load	Φόρτωση δεδομένων CSV/TSV
POST	/api/projects/{id}/loadSchemaEvo	Φόρτωση δεδομένων Schema Evolution από φάκελο
GET	/api/projects/{id}/info	Ανάκτηση πληροφοριών project
POST	/api/projects/{id}/cluster	Εφαρμογή clustering
GET	/api/projects/{id}/chartData	Ανάκτηση δεδομένων για PLD
POST	/api/projects/{id}/sort	Ταξινόμηση οντοτήτων
GET	/api/projects/{id}/patterns	Ανίχνευση και ανάκτηση μοτίβων
POST	/api/projects/{id}/exportProject	Εξαγωγή project σε φάκελο
POST	/api/projects/{id}/importProject	Εισαγωγή project από φάκελο
POST	/api/projects/{id}/saveData	Εξαγωγή δεδομένων σε αρχεία TSV
GET	/api/projects/{id}/cellDetails	Ανάκτηση λεπτομερειών κελιού PLD

Κάθε endpoint λειτουργεί πάνω σε ένα ενεργό project, το οποίο ταυτοποιείται μέσω μοναδικού αναγνωριστικού (id) και αντιστοιχεί σε ένα session χρήστη.

4.4 Διαχείριση Συνεδριών (Session Management)

Η διαχείριση της κατάστασης των χρηστών υλοποιείται μέσω της κλάσης ProjectStore, η οποία λειτουργεί ως μηχανισμός αποθήκευσης ενεργών projects στη μνήμη του server. Η κλάση αυτή αποτελεί κεντρικό component της αρχιτεκτονικής, καθώς διαχειρίζεται τον κύκλο ζωής κάθε project από τη δημιουργία μέχρι τη διαγραφή του.

Κάθε project που δημιουργείται αντιστοιχεί σε μια μοναδική συνεδρία χρήστη. Όταν ένας χρήστης ανοίγει την εφαρμογή, δημιουργείται αυτόματα ένα νέο project με μοναδικό UUID αναγνωριστικό. Αυτό το UUID χρησιμοποιείται σε όλες τις επόμενες κλήσεις API για την ταυτοποίηση του project στο οποίο αναφέρεται κάθε ενέργεια. Με αυτόν τον τρόπο, πολλαπλοί χρήστες μπορούν να εργάζονται ταυτόχρονα σε διαφορετικά projects χωρίς παρεμβολές.

Τα βασικά χαρακτηριστικά της διαχείρισης συνεδριών είναι:

- **Thread-safe αποθήκευση:** Η χρήση ConcurrentHashMap [9] εξασφαλίζει ότι η πρόσβαση και η τροποποίηση των δεδομένων είναι ασφαλής σε περιβάλλον με πολλαπλά threads. Αυτό είναι κρίσιμο για τη σωστή λειτουργία σε web server όπου κάθε αίτημα HTTP μπορεί να εξυπηρετηθεί από διαφορετικό thread.
- **Αυτόματη λήξη συνεδριών:** Κάθε project διαθέτει timestamp τελευταίας πρόσβασης (lastAccessed). Αν ένα project παραμείνει ανενεργό για περισσότερο από 30 λεπτά, θεωρείται expired και διαγράφεται αυτόματα. Αυτός ο μηχανισμός αποτρέπει τη διαρροή μνήμης από εγκαταλελειμμένες συνεδρίες.
- **Περιοδικός καθαρισμός:** Ένα scheduled task εκτελείται κάθε 5 λεπτά και εντοπίζει τα expired projects, απελευθερώνοντας τη μνήμη που καταλαμβάνουν. Αυτό εξασφαλίζει τη σταθερή λειτουργία του server ακόμα και μετά από μακροχρόνια χρήση.
- **Όριο ταυτόχρονων συνεδριών:** Υλοποιήθηκε μηχανισμός που περιορίζει τον μέγιστο αριθμό ταυτόχρονων projects σε 100, προστατεύοντας τον server από υπερβολική χρήση πόρων. Όταν επιτευχθεί το όριο, οι νέες αιτήσεις δημιουργίας project απορρίπτονται με κατάλληλο μήνυμα σφάλματος.
- **SendBeacon Support:** Υποστηρίζεται το Browser API sendBeacon για τον καθαρισμό projects κατά το κλείσιμο της σελίδας. Αυτό εξασφαλίζει ότι τα projects διαγράφονται αμέσως όταν ο χρήστης φεύγει, αντί να περιμένουν τη λήξη από timeout.

Η προσέγγιση αυτή επιτρέπει την ταυτόχρονη εξυπηρέτηση πολλαπλών χρηστών, χωρίς ανάγκη μόνιμης αποθήκευσης δεδομένων σε βάση. Η απουσία εξωτερικής βάσης δεδομένων απλοποιεί τη διαχείριση και την ανάπτυξη του συστήματος, καθιστώντας την εφαρμογή εύκολα εγκαταστάσιμη σε οποιοδήποτε περιβάλλον με Java runtime.

Όλες οι αποκρίσεις σφάλματος ακολουθούν τυποποιημένη δομή JSON με τα πεδία: success (πάντα false), errorCode (αλφαριθμητικός κωδικός), error (αναγνώσιμο μήνυμα), και status (HTTP status code). Επιπλέον, κάθε σφάλμα καταγράφεται μέσω

του SLF4J Logger με κατάλληλο επίπεδο (warn για αναμενόμενα σφάλματα, error για μη αναμενόμενα).

4.4.1 Διαχείριση Σφαλμάτων (GlobalExceptionHandler)

Για την ομοιόμορφη διαχείριση σφαλμάτων σε όλα τα endpoints του REST API υλοποιήθηκε η κλάση GlobalExceptionHandler, η οποία χρησιμοποιεί το @ControllerAdvice annotation του Spring Framework [2] για την κεντρική διαχείριση εξαιρέσεων.

Η κλάση παρέχει εξειδικευμένους handlers για τους ακόλουθους τύπους σφαλμάτων:

- **Exception (generic):** Για όλες τις μη αναμενόμενες εξαιρέσεις επιστρέφει HTTP 500 με κωδικό INTERNAL_ERROR και γενικό μήνυμα, αποτρέποντας τη διαρροή τεχνικών λεπτομερειών στον client.
- **ProjectNotFoundException:** Επιστρέφει HTTP 404 (Not Found) όταν ζητείται ανύπαρκτο project, με κωδικό PROJECT_NOT_FOUND. Η ProjectNotFoundException είναι custom exception class ορισμένη ως εσωτερική κλάση του GlobalExceptionHandler.
- **IllegalArgumentException:** Επιστρέφει HTTP 400 (Bad Request) για σφάλματα επικύρωσης παραμέτρων, με κωδικό VALIDATION_ERROR.
- **IOException:** Επιστρέφει HTTP 500 (Internal Server Error) για σφάλματα εισόδου/εξόδου κατά την επεξεργασία αρχείων, με κωδικό FILE_ERROR και λεπτομερές μήνυμα.
- **MaxUploadSizeExceededException:** Επιστρέφει HTTP 413 (Payload Too Large) όταν το μέγεθος του αρχείου υπερβαίνει το όριο των 100MB, με κωδικό σφάλματος FILE_TOO_LARGE.

4.5 Frontend

Το frontend της εφαρμογής υλοποιήθηκε ως Single Page Application (SPA) και βασίζεται αποκλειστικά σε τεχνολογίες web, χωρίς εξωτερικά frameworks ή βιβλιοθήκες. Η επιλογή αυτή έγινε για να διατηρηθεί η απλότητα της αρχιτεκτονικής και να αποφευχθούν εξαρτήσεις που θα μπορούσαν να επηρεάσουν τη μακροπρόθεσμη συντηρησιμότητα. Οι βασικές τεχνολογίες που χρησιμοποιήθηκαν είναι:

HTML5 [11]**:** Χρησιμοποιείται για τη δομή της σελίδας και τον ορισμό των βασικών components. Η σημασιολογική δομή του HTML5 βοηθά στην οργάνωση του κώδικα και την προσβασιμότητα. Τα βασικά στοιχεία περιλαμβάνουν: το toolbar με τα κουμπιά ελέγχου, τον container του Parallel Lives Diagram, τα modal dialogs για τις ρυθμίσεις clustering και τις πληροφορίες κελιών, και τα side panels για τις επιλογές aggregation.

CSS3: Χρησιμοποιείται για τη μορφοποίηση και την αισθητική του περιβάλλοντος. Υιοθετήθηκαν σχεδιαστικές αρχές glassmorphism για ένα σύγχρονο και ελκυστικό visual design, με χρήση ημιδιαφανών backgrounds, blur effects, και subtle shadows. Χρησιμοποιούνται CSS Grid και Flexbox για responsive layout, CSS transitions για ομαλές μεταβάσεις, και CSS variables για εύκολη παραμετροποίηση χρωμάτων και διαστάσεων.

JavaScript [4] **(ES6+):** Χρησιμοποιείται για την επιχειρησιακή λογική του frontend και την επικοινωνία με το REST API μέσω του Fetch API. Ο κώδικας είναι οργανωμένος σε modules με σαφή διαχωρισμό ευθυνών: ο ApiService χειρίζεται τις κλήσεις REST, ο ChartRenderer σχεδιάζει το Parallel Lives Diagram, ο UIManager διαχειρίζεται τα dialogs και τα events, και ο StateManager διατηρεί την τρέχουσα κατάσταση της εφαρμογής. Χρησιμοποιούνται modern JavaScript features όπως async/await, destructuring, template literals, και arrow functions.

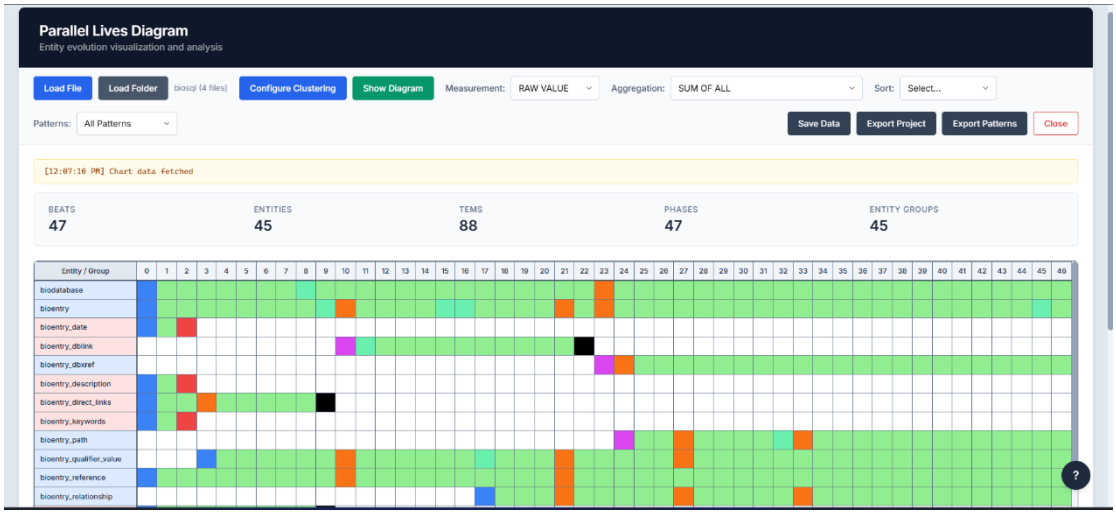
Η αρχιτεκτονική SPA επιτρέπει δυναμική ενημέρωση του περιβάλλοντος χωρίς επαναφόρτωση της σελίδας, βελτιώνοντας σημαντικά την εμπειρία χρήσης [4]. Η εφαρμογή φορτώνεται μία φορά στον browser και στη συνέχεια όλες οι αλληλεπιδράσεις γίνονται μέσω ασύγχρονων κλήσεων AJAX προς το REST API. Τα αποτελέσματα ενημερώνουν δυναμικά το DOM χωρίς να διακόπτεται η ροή εργασίας του χρήστη.

Το Parallel Lives Diagram απεικονίζεται ως HTML table που δημιουργείται δυναμικά με JavaScript. Κάθε κελί του πίνακα αντιστοιχεί σε ένα GroupPhaseMeasurement (GPM) και χρωματίζεται ανάλογα με την τιμή του χρησιμοποιώντας μια κλίμακα χρωμάτων (color scale) που μεταβάλλεται από ανοιχτό (χαμηλές τιμές) σε σκούρο (υψηλές τιμές). Τα κελιά που περιέχουν births, deaths, ή ανιχνευμένα patterns επισημαίνονται με ειδικούς δείκτες (icons) που επιτρέπουν την άμεση αναγνώρισή τους.

4.5.1 Γενική Επισκόπηση Διεπαφής Χρήστη

Η κεντρική οθόνη της εφαρμογής αποτελείται από τρεις βασικές περιοχές: την επάνω γραμμή εργαλείων (toolbar) που περιέχει τα κουμπιά ελέγχου και τις επιλογές εξαγωγής, τον κεντρικό χώρο απεικόνισης του Parallel Lives Diagram (PLD), και τα

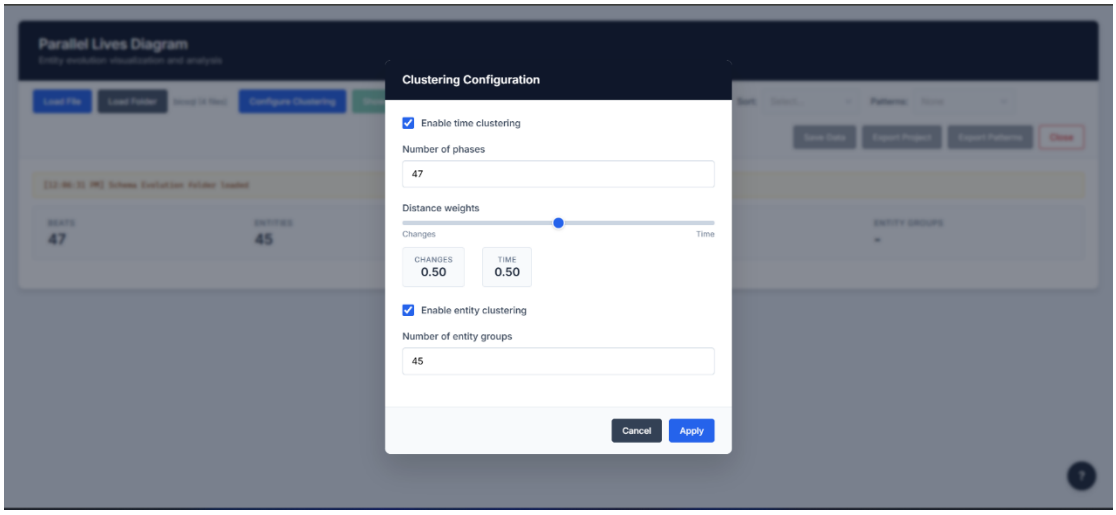
πλαϊνά panels για τη διαμόρφωση του clustering και των aggregation options. Η διάταξη σχεδιάστηκε με γνώμονα την εργονομία και την άμεση πρόσβαση στις βασικές λειτουργίες, επιτρέποντας στον χρήστη να εκτελεί την πλήρη ροή εργασίας χωρίς περιττές μετακινήσεις.



Εικόνα 3: Γενική Επισκόπηση Διεπαφής Χρήστη

4.5.2 Παράθυρο Ρύθμισης Clustering (Configure Clustering)

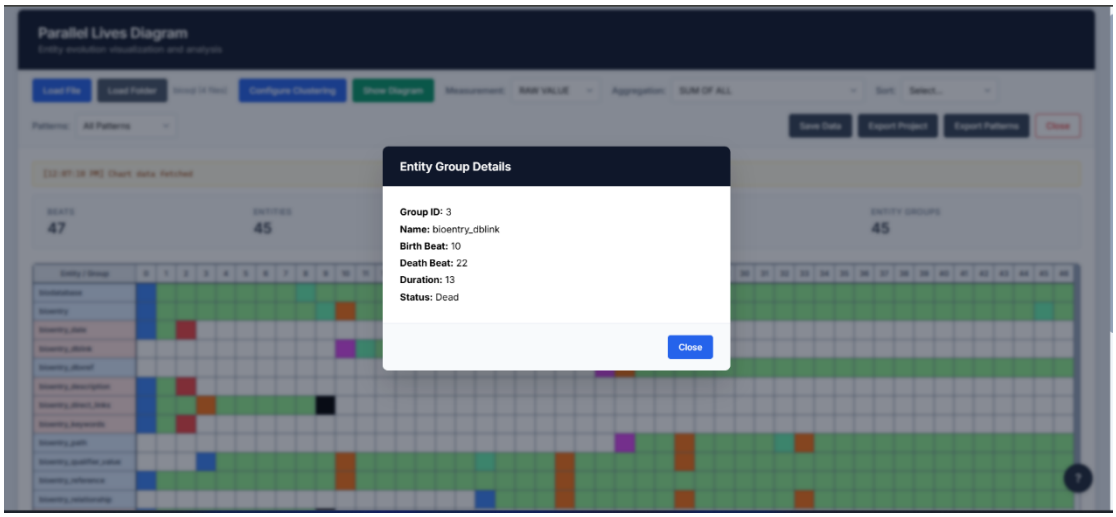
Το παράθυρο Configure Clustering επιτρέπει στον χρήστη να διαμορφώσει τις παραμέτρους ομαδοποίησης των δεδομένων. Περιλαμβάνει δύο βασικές ενότητες: την ομαδοποίηση χρονικών διαστημάτων (Time Clustering) και την ομαδοποίηση οντοτήτων (Entity Clustering). Για κάθε τύπο ομαδοποίησης, ο χρήστης μπορεί να επιλέξει τον επιθυμητό αριθμό ομάδων μέσω slider ή απευθείας εισαγωγής αριθμού. Επιπλέον, διατίθεται η ρύθμιση Distance Weights, η οποία καθορίζει τη σχετική βαρύτητα μεταξύ των αλλαγών (Changes) και του χρόνου (Time) κατά τον υπολογισμό της απόστασης μεταξύ διαδοχικών beats στον αλγόριθμο Agglomerative Clustering. Όταν το slider βρίσκεται προς τα αριστερά (100% Changes), ο αλγόριθμος ομαδοποιεί beats με παρόμοιο αριθμό αλλαγών ανεξάρτητα από τη χρονική τους απόσταση, ενώ όταν βρίσκεται προς τα δεξιά (100% Time), ομαδοποιεί beats που είναι χρονικά κοντά ανεξάρτητα από τον αριθμό αλλαγών τους.



Εικόνα 4: Παράθυρο Ρύθμισης Clustering (Configure Clustering)

4.5.3 Παράθυρο Πληροφοριών Οντότητας (Entity Info)

Κατά το κλικ στο όνομα μιας οντότητας ή ομάδας οντοτήτων στον πίνακα, εμφανίζεται ένα αναδυόμενο παράθυρο με αναλυτικές πληροφορίες. Το παράθυρο αυτό παρουσιάζει τα ονόματα όλων των οντοτήτων που περιλαμβάνονται στην ομάδα, τις χρονικές στιγμές γέννησης (birth beat) και θανάτου (death beat) κάθε οντότητας, καθώς και στατιστικά στοιχεία όπως η συνολική διάρκεια ζωής. Η λειτουργία αυτή διευκολύνει τον χρήστη να κατανοήσει τη σύνθεση των ομάδων μετά από clustering και να εξετάσει τα χαρακτηριστικά μεμονωμένων οντοτήτων.



Εικόνα 5: Παράθυρο Πληροφοριών Οντότητας (Entity Info)

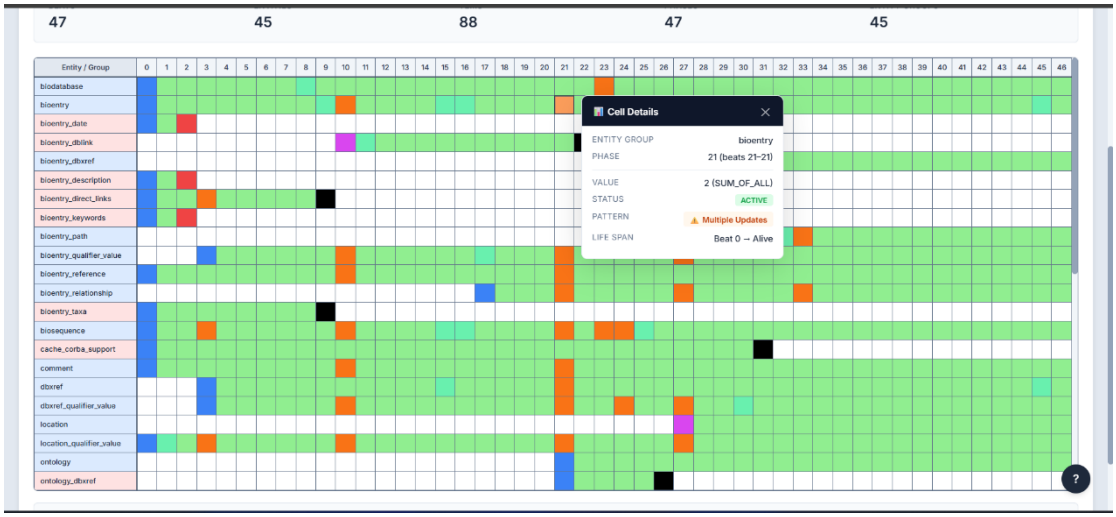
4.5.4 Διαδραστικά Κελιά και Λεπτομέρειες Κελιού (Cell Details)

Μια από τις νέες λειτουργίες που υλοποιήθηκαν στην παρούσα εργασία είναι η δυνατότητα προβολής αναλυτικών πληροφοριών για κάθε κελί του Parallel Lives

Diagram. Με κλικ σε οποιοδήποτε κελί του πίνακα, εμφανίζεται ένα αναδυόμενο παράθυρο που παρουσιάζει:

- **Entity Group:** Τα ονόματα των οντοτήτων που ανήκουν στην ομάδα αυτού του κελιού.
- **Phase:** Η φάση (ομάδα διαδοχικών beats) και το εύρος των beats που περιλαμβάνει.
- **Value:** Η συγκεντρωτική τιμή του κελιού (π.χ. άθροισμα αλλαγών) και ο τύπος aggregation που εφαρμόστηκε.
- **Status:** Η κατάσταση της οντότητας στη συγκεκριμένη φάση (Birth, Death, Active, Inactive).
- **Pattern:** Αν το κελί ανήκει σε κάποιο ανιχνευμένο pattern, εμφανίζεται ο τύπος του.
- **Life Span:** Οι χρονικές στιγμές γέννησης και θανάτου της οντότητας/ομάδας.
- **Entity Breakdown:** Αναλυτικές τιμές για κάθε οντότητα ξεχωριστά (όταν η ομάδα περιέχει περισσότερες από μία οντότητες).

Η υλοποίηση αυτής της λειτουργίας βασίζεται στα δεδομένα που επιστρέφονται από το endpoint GET /api/projects/{id}/chartData. Η δομή δεδομένων ChartGroupPhaseMeasurement περιλαμβάνει όλα τα απαραίτητα πεδία για κάθε κελί του πίνακα: entityGroupId, entityGroupName, phaseId, value, status (GPMType: Birth, Death, Active, Inactive), και πληροφορίες μοτίβων. Κατά το κλικ σε ένα κελί, ο JavaScript κώδικας ανακτά τα αντίστοιχα στοιχεία από την ήδη φορτωμένη δομή δεδομένων και τα εμφανίζει σε tooltip, χωρίς να απαιτείται νέο HTTP αίτημα προς τον server. Η σχεδίαση αυτή ακολουθεί την αρχή της ελάχιστης επέμβασης στον υπάρχοντα κώδικα, αξιοποιώντας τα διαθέσιμα δεδομένα αντί να προσθέτει νέα endpoints.

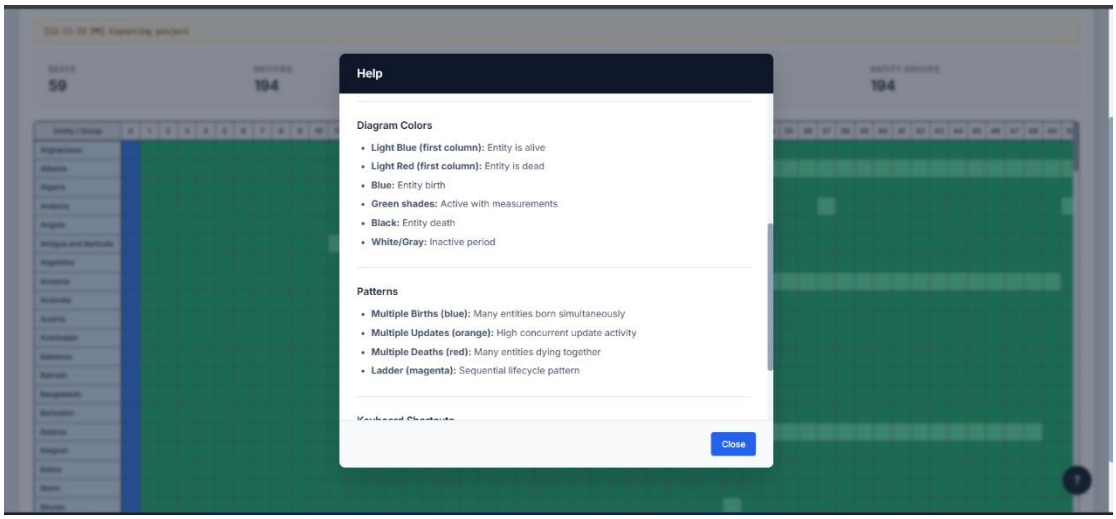


Εικόνα 6: Παράθυρο Λεπτομερειών Κελιού (Cell Details)

4.5.5 Παράθυρο Βοήθειας (Help)

Η εφαρμογή παρέχει ενσωματωμένο σύστημα βοήθειας προσβάσιμο μέσω του κουμπιού "Help" στη γραμμή εργαλείων ή πατώντας το πλήκτρο "?" από το πληκτρολόγιο. Το παράθυρο βοήθειας εμφανίζεται ως modal dialog και περιέχει όλες τις απαραίτητες πληροφορίες για την αποτελεσματική χρήση της εφαρμογής, οργανωμένες σε πέντε θεματικές ενότητες.

Η πρώτη ενότητα (Supported Input Formats) περιγράφει τους υποστηριζόμενους τύπους αρχείων εισόδου, συμπεριλαμβανομένων αρχείων TSV/CSV και φακέλων με δεδομένα Schema Evolution ή προ-επεξεργασμένα projects. Η δεύτερη ενότητα (Quick Start) παρέχει συνοπτικές οδηγίες για τη βασική ροή εργασίας: φόρτωση δεδομένων, ρύθμιση clustering, και οπτικοποίηση. Η τρίτη ενότητα (Diagram Colors) εξηγεί τη σημασία των χρωμάτων στον πίνακα: μπλε για births, πράσινες αποχρώσεις για ενεργές οντότητες με μετρήσεις, μαύρο για deaths, και λευκό/γκρι για ανενεργές περιόδους. Η τέταρτη ενότητα (Patterns) περιγράφει τα ανιχνεύσιμα μοτίβα (Multiple Births, Multiple Updates, Multiple Deaths, Ladder) και την οπτική τους αναπαράσταση. Τέλος, η πέμπτη ενότητα (Keyboard Shortcuts) παραθέτει τις διαθέσιμες συντομεύσεις πληκτρολογίου για γρήγορη πρόσβαση σε συχνές λειτουργίες όπως αποθήκευση (Ctrl+S), εξαγωγή (Ctrl+E), και κλείσιμο παραθύρων (Escape).



Εικόνα 7: Παράθυρο Βοήθειας (Help Modal)

4.6 Νέα Αρχεία Υλοποίησης

Στον παρακάτω πίνακα συνοψίζονται τα βασικά νέα αρχεία που προστέθηκαν στο πλαίσιο της παρούσας εργασίας:

Αρχείο	**Γραμμές Κώδικα**	**Ρόλος**
DaintinessRestAPI.java	~925	Υλοποίηση REST Controller
ProjectStore.java	~125	Διαχείριση συνεδριών και projects
GlobalExceptionHandler.java	~85	Κεντρικός χειρισμός σφαλμάτων
index.html	~2570	Frontend Single Page Application

Κεφάλαιο 5. Έλεγχος

5.1 Εισαγωγή

Στο παρόν κεφάλαιο παρουσιάζεται η διαδικασία ελέγχου και επαλήθευσης της web εφαρμογής Plutarch Parallel Lives. Ο έλεγχος επικεντρώνεται τόσο στη σωστή λειτουργία του REST API όσο και στη διαχείριση των συνεδριών χρηστών.

Συνολικά υλοποιήθηκαν 49 tests, εκ των οποίων 24 integration tests για το REST API, 25 unit tests για τον ProjectStore, με στόχο τη διασφάλιση της αξιοπιστίας και ορθότητας του συστήματος. Η εκτέλεση των tests πραγματοποιείται αυτόματα μέσω του Maven για τα Java tests.

5.2 Έλεγχος REST API

Ο έλεγχος του REST API πραγματοποιήθηκε μέσω της κλάσης DaintinessRestAPITest, η οποία περιλαμβάνει 24 integration tests [3]. Τα tests αυτά ελέγχουν τη σωστή συνεργασία όλων των επιπέδων του συστήματος, από τη λήψη του HTTP αιτήματος έως την απόκριση του backend. Κάθε test δημιουργεί ένα νέο project, εκτελεί τις απαραίτητες ενέργειες και επαληθεύει τα αναμενόμενα αποτελέσματα, ενώ στο τέλος καθαρίζει τους πόρους που δημιούργησε.

Τα integration tests οργανώνονται σε κατηγορίες ανάλογα με τη λειτουργικότητα που ελέγχουν. Κάθε κατηγορία καλύπτει τόσο τις επιτυχημένες περιπτώσεις (happy path) όσο και τις περιπτώσεις σφαλμάτων (error cases), διασφαλίζοντας την ορθή συμπεριφορά του συστήματος υπό όλες τις συνθήκες.

5.2.1 Διαχείριση Projects (Project Management Tests)

Η πρώτη κατηγορία tests αφορά τη δημιουργία, ανάκτηση και διαγραφή projects. Αυτά τα tests αποτελούν τη βάση για όλες τις υπόλοιπες λειτουργίες, καθώς κάθε ενέργεια στο σύστημα απαιτεί ένα ενεργό project.

createProject_ShouldReturnProjectId: Επαληθεύει ότι η κλήση POST /api/projects δημιουργεί επιτυχώς νέο project και επιστρέφει μοναδικό αναγνωριστικό (UUID). Ελέγχει επίσης ότι το project είναι προσβάσιμο μέσω του ProjectStore.

createProject_MaxSessionsReached_ShouldReturn503: Επαληθεύει ότι όταν το σύστημα φτάσει το μέγιστο αριθμό ενεργών συνεδριών (maxSessions), η δημιουργία νέου project επιστρέφει HTTP 503 Service Unavailable με κατάλληλο μήνυμα σφάλματος.

5.2.2 Φόρτωση Δεδομένων (Data Loading Tests)

Η δεύτερη κατηγορία tests αφορά τη φόρτωση αρχείων δεδομένων. Ελέγχονται τόσο η επιτυχής φόρτωση αρχείων TSV όσο και οι περιπτώσεις σφαλμάτων.

loadFile_ValidTsv_ShouldSucceed: Επαληθεύει ότι η φόρτωση έγκυρου αρχείου TSV ολοκληρώνεται επιτυχώς. Ελέγχει ότι η απόκριση περιλαμβάνει τον αριθμό beats και entities που φορτώθηκαν, επιβεβαιώνοντας την ορθή ανάγνωση του αρχείου.

loadFile_NonExistentProject_ShouldReturn404: Επαληθεύει ότι η προσπάθεια φόρτωσης αρχείου σε ανύπαρκτο project επιστρέφει κωδικό σφάλματος 404.

5.2.3 Ομαδοποίηση (Clustering Tests)

Η τρίτη κατηγορία tests αφορά την εφαρμογή clustering στα δεδομένα. Ελέγχονται οι έγκυρες παράμετροι, οι μη έγκυρες τιμές, καθώς και η ορθή εκτέλεση του αλγορίθμου Agglomerative Clustering.

clusterData_ValidRequest_ShouldSucceed: Επαληθεύει ότι η εφαρμογή clustering με έγκυρες παραμέτρους (αριθμός φάσεων, αριθμός ομάδων οντοτήτων, βάρος αλλαγών) ολοκληρώνεται επιτυχώς και επιστρέφει τον πραγματικό αριθμό φάσεων και ομάδων που δημιουργήθηκαν.

clusterData_InvalidPhases_ShouldReturn400: Επαληθεύει ότι η προσπάθεια clustering με μη έγκυρο αριθμό φάσεων (π.χ. 0 ή αρνητικό) απορρίπτεται με κωδικό σφάλματος 400 (Bad Request).

clusterData_InvalidEntityGroups_ShouldReturn400: Επαληθεύει ότι η προσπάθεια clustering με numberOfEntityGroups = 0 επιστρέφει HTTP 400 με success=false. Επιπλέον, οι έλεγχοι clusterData_InvalidChangesWeight_ShouldReturn400 και clusterData_ChangesWeightTooHigh_ShouldReturn400 επαληθεύουν ότι μη έγκυρες τιμές changesWeight (αρνητικές τιμές ή τιμές > 1.0) επιστρέφουν HTTP 400.

5.2.4 Ανάκτηση Δεδομένων Πίνακα (Chart Data Tests)

Η τέταρτη κατηγορία tests αφορά την ανάκτηση δεδομένων για την απεικόνιση του Parallel Lives Diagram. Ελέγχεται η σωστή δομή της απόκρισης και η συμπεριφορά πριν και μετά το clustering.

getChartData_AfterClustering_ShouldSucceed: Επαληθεύει ότι μετά την εκτέλεση clustering, η κλήση GET /api/projects/{id}/chartData επιστρέφει πλήρη δεδομένα με τις απαραίτητες δομές: chartData (πίνακας μετρήσεων), phases (λίστα φάσεων), και entityGroups (λίστα ομάδων οντοτήτων).

getChartData_BeforeClustering_ShouldReturn400: Επαληθεύει ότι η προσπάθεια ανάκτησης δεδομένων πίνακα πριν την εκτέλεση clustering επιστρέφει HTTP 400 με κατάλληλο μήνυμα σφάλματος. Επιπλέον, ο έλεγχος getChartData_NonExistentProject_ShouldReturn404 επαληθεύει ότι η πρόσβαση σε ανύπαρκτο project επιστρέφει HTTP 404.

5.2.5 Ταξινόμηση και Μοτίβα (Sorting and Patterns Tests)

Η πέμπτη κατηγορία tests αφορά την ταξινόμηση οντοτήτων και την ανίχνευση μοτίβων στα δεδομένα.

sortChartData_ValidSortType_ShouldSucceed: Επαληθεύει ότι η ταξινόμηση οντοτήτων με έγκυρο τύπο ταξινόμησης (π.χ. BIRTH_ASCENDING, LIFE_DURATION_DESCENDING) εκτελείται επιτυχώς.

sortChartData_InvalidSortType_ShouldReturn400: Επαληθεύει ότι η προσπάθεια ταξινόμησης με μη έγκυρο τύπο επιστρέφει HTTP 400. Επιπλέον, ο έλεγχος sortChartData_BeforeClustering_ShouldReturn400 επαληθεύει ότι η ταξινόμηση πριν το clustering επιστρέφει σφάλμα.

getPatterns_ValidType_ShouldReturnList: Επαληθεύει ότι η κλήση GET /api/projects/{id}/patterns επιστρέφει λίστα μοτίβων. Επιπλέον, ο έλεγχος

getPatterns_InvalidType_ShouldReturn400 επαληθεύει ότι μη έγκυρος τύπος μοτίβου (π.χ. INVALID_PATTERN_TYPE) επιστρέφει HTTP 400.

5.2.6 Λεπτομέρειες Κελιού (Cell Details Tests)

Η έκτη κατηγορία tests αφορά τη λειτουργία ανάκτησης λεπτομερειών για συγκεκριμένο κελί του Parallel Lives Diagram μέσω του endpoint GET /api/projects/{id}/cellDetails.

getCellDetails_ValidRequest_ShouldSucceed: Επαληθεύει ότι η κλήση με έγκυρα entityGroupId και phaseId επιστρέφει πλήρεις πληροφορίες κελιού, συμπεριλαμβανομένων: ονομάτων οντοτήτων, εύρους φάσης, τιμής μέτρησης, κατάστασης (birth/death/active/inactive), και αναλυτικής κατανομής τιμών ανά οντότητα (breakdown).

getCellDetails_NonExistentProject_ShouldReturn404: Επαληθεύει ότι η προσπάθεια ανάκτησης λεπτομερειών για ανύπαρκτο project επιστρέφει HTTP 404.

getCellDetails_InvalidIndices_ShouldReturn400: Επαληθεύει ότι μη έγκυροι δείκτες (π.χ. entityGroupId=999) επιστρέφουν HTTP 400 με κατάλληλο μήνυμα σφάλματος.

getCellDetails_BeforeClustering_ShouldReturn400: Επαληθεύει ότι η ανάκτηση λεπτομερειών πριν την εκτέλεση clustering επιστρέφει HTTP 400.

5.2.7 Εξαγωγή και Αποθήκευση (Export Tests)

Η έβδομη κατηγορία tests αφορά τις λειτουργίες εξαγωγής δεδομένων και αποθήκευσης projects.

exportProject_AfterClustering_ShouldSucceed: Επαληθεύει ότι η εξαγωγή project μετά το clustering επιστρέφει αρχείο ZIP με τα απαραίτητα headers (Content-Disposition: attachment).

exportProject_BeforeClustering_ShouldReturn400: Επαληθεύει ότι η προσπάθεια εξαγωγής project πριν την εκτέλεση clustering απορρίπτεται, καθώς δεν υπάρχουν ακόμα δεδομένα προς εξαγωγή.

saveData_WithLoadedData_ShouldSucceed: Επαληθεύει ότι η αποθήκευση δεδομένων σε μορφή TSV ολοκληρώνεται επιτυχώς και επιστρέφει το αρχείο με τα κατάλληλα headers.

5.2.8 Στατιστικά Συνεδριών (Session Statistics Tests)

Η τελευταία κατηγορία tests αφορά την παρακολούθηση ενεργών συνεδριών στο σύστημα.

getSessionStats_ShouldReturnCorrectCount: Επαληθεύει ότι η κλήση GET /api/sessions/stats επιστρέφει σωστό αριθμό ενεργών projects. Ο έλεγχος δημιουργεί 2 projects και επαληθεύει ότι το activeProjects είναι τουλάχιστον 2, διασφαλίζοντας ότι το endpoint παρέχει ακριβή στατιστικά.

Κατηγορία	**Αριθμός Tests**	**Περιγραφή**
Project Management	2	Δημιουργία νέου project και έλεγχος ορίου μέγιστων συνεδριών.
Data Loading	2	Φόρτωση έγκυρου αρχείου TSV και έλεγχος ανύπαρκτου project.
Clustering	5	Εκτέλεση clustering και έλεγχοι μη έγκυρων παραμέτρων.
Chart Data	3	Ανάκτηση δεδομένων διαγράμματος πριν και μετά το clustering.
Sorting & Patterns	4	Ταξινόμηση οντοτήτων και ανάκτηση μοτίβων δεδομένων.
Cell Details	4	Ανάκτηση λεπτομερειών κελιού και έλεγχος σφαλμάτων.
Export & Save	3	Εξαγωγή project και αποθήκευση δεδομένων.
Session Statistics	1	Έλεγχος αριθμού ενεργών συνεδριών.

Τα integration tests επιβεβαιώνουν ότι τα endpoints ανταποκρίνονται σωστά σε έγκυρα και μη έγκυρα αιτήματα και ότι η ροή δεδομένων μεταξύ frontend, REST API και backend λειτουργεί όπως αναμένεται. Η επιτυχής εκτέλεση όλων των tests διασφαλίζει την αξιοπιστία και τη σταθερότητα της web εφαρμογής.

5.3 Έλεγχος ProjectStore

Η κλάση ProjectStoreTest περιλαμβάνει 25 unit tests, τα οποία εστιάζουν αποκλειστικά στη λειτουργικότητα του in-memory session store, ανεξάρτητα από το REST API. Τα tests οργανώνονται σε κατηγορίες ανάλογα με τη μέθοδο που ελέγχουν.

5.3.1 Δημιουργία Project (createProject Tests)

Η πρώτη κατηγορία περιλαμβάνει 5 tests που ελέγχουν τη δημιουργία νέων projects.

createProject_ShouldReturnNonNullId: Επαληθεύει ότι η δημιουργία project επιστρέφει μη-null αναγνωριστικό.

createProject_ShouldReturnValidUUID: Επαληθεύει ότι το επιστρεφόμενο ID έχει έγκυρη μορφή UUID.

createProject_ShouldReturnUniqueIds: Επαληθεύει ότι διαδοχικές δημιουργίες επιστρέφουν μοναδικά IDs.

createProject_ShouldStoreForRetrieval: Επαληθεύει ότι το δημιουργημένο project είναι ανακτήσιμο μέσω του ID του.

createProject_ShouldIncrementActiveCount: Επαληθεύει ότι ο μετρητής ενεργών projects αυξάνεται κατά τη δημιουργία.

5.3.2 Ανάκτηση Project (get Tests)

Η δεύτερη κατηγορία περιλαμβάνει 5 tests που ελέγχουν την ανάκτηση projects από το store.

get_ShouldReturnEmptyForNonExistent: Επαληθεύει ότι η αναζήτηση ανύπαρκτου project επιστρέφει empty Optional.

get_ShouldReturnEmptyForNullId: Επαληθεύει ότι η αναζήτηση με null ID επιστρέφει empty Optional.

get_ShouldReturnEmptyForEmptyId: Επαληθεύει ότι η αναζήτηση με κενό string ID επιστρέφει empty Optional.

get_ShouldReturnSameControllerInstance: Επαληθεύει ότι επαναλαμβανόμενες κλήσεις get για το ίδιο project επιστρέφουν το ίδιο controller instance.

get_ShouldReturnDifferentControllers: Επαληθεύει ότι διαφορετικά projects έχουν διαφορετικά controller instances.

5.3.3 Διαγραφή Project (delete Tests)

Η τρίτη κατηγορία περιλαμβάνει 7 tests που ελέγχουν τη διαγραφή projects.

delete_ShouldReturnTrueForExisting: Επαληθεύει ότι η διαγραφή υπάρχοντος project επιστρέφει true.

delete_ShouldReturnFalseForNonExistent: Επαληθεύει ότι η διαγραφή ανύπαρκτου project επιστρέφει false.

delete_ShouldReturnFalseForNullId: Επαληθεύει ότι η διαγραφή με null ID επιστρέφει false.

delete_ShouldRemoveFromStore: Επαληθεύει ότι μετά τη διαγραφή το project δεν είναι πλέον ανακτήσιμο.

delete_ShouldDecrementActiveCount: Επαληθεύει ότι ο μετρητής ενεργών projects μειώνεται κατά τη διαγραφή.

delete_ShouldReturnFalseWhenDeletingTwice: Επαληθεύει ότι η δεύτερη διαγραφή του ίδιου project επιστρέφει false.

delete_ShouldNotAffectOtherProjects: Επαληθεύει ότι η διαγραφή ενός project δεν επηρεάζει άλλα projects.

5.3.4 Μέτρηση Ενεργών Projects (getActiveProjectCount Tests)

Η τέταρτη κατηγορία περιλαμβάνει 2 tests που ελέγχουν τη μέτρηση ενεργών sessions.

getActiveProjectCount_ShouldReturnZeroForEmptyStore: Επαληθεύει ότι ο μετρητής επιστρέφει 0 για άδειο store.

getActiveProjectCount_ShouldTrackCorrectly: Επαληθεύει ότι ο μετρητής ενημερώνεται σωστά μετά από δημιουργίες και διαγραφές.

5.3.5 Καθαρισμός Συνεδριών (Session Cleanup Tests)

Η πέμπτη κατηγορία περιλαμβάνει 2 tests που ελέγχουν τον μηχανισμό αυτόματου καθαρισμού.

cleanupExpiredSessions_ShouldNotRemoveActive: Επαληθεύει ότι ο καθαρισμός δεν αφαιρεί ενεργές συνεδρίες που δεν έχουν λήξει.

forceCleanup_ShouldReturnRemovedCount: Επαληθεύει ότι ο καθαρισμός επιστρέφει τον αριθμό των αφαιρεθέντων ληγμένων συνεδριών.

5.3.6 Ασφάλεια Νημάτων (Thread Safety Tests)

Η έκτη κατηγορία περιλαμβάνει 2 tests που ελέγχουν την ασφαλή ταυτόχρονη πρόσβαση.

shouldHandleConcurrentCreation: Επαληθεύει ότι πολλαπλά threads μπορούν να δημιουργήσουν projects ταυτόχρονα χωρίς race conditions.

shouldHandleConcurrentReadAndDelete: Επαληθεύει ότι ταυτόχρονες αναγνώσεις και διαγραφές δεν προκαλούν προβλήματα.

5.3.7 Ολοκληρωμένος Κύκλος Ζωής (Lifecycle Tests)

Η τελευταία κατηγορία περιλαμβάνει 2 tests που ελέγχουν τον πλήρη κύκλο ζωής.

fullLifecycle_CreateGetDelete: Επαληθεύει τη σωστή λειτουργία της ακολουθίας δημιουργία → ανάκτηση → διαγραφή.

controllerShouldBeProperlyInitialized: Επαληθεύει ότι ο controller που δημιουργείται είναι σωστά αρχικοποιημένος και έτοιμος για χρήση.

Κατηγορία	**Αριθμός Tests**	**Περιγραφή**
Create & Get	5	Δημιουργία και ανάκτηση projects
Remove	4	Διαγραφή projects
Touch	4	Ενημέρωση χρόνου τελευταίας πρόσβασης
Expiration	5	Αυτόματη λήξη ανενεργών projects

Statistics	3	Έλεγχος στατιστικών στοιχείων
Thread Safety	4	Έλεγχος ταυτόχρονης πρόσβασης (concurrent access)

Τα unit tests του ProjectStore διασφαλίζουν ότι ο μηχανισμός διαχείρισης συνεδριών λειτουργεί αξιόπιστα τόσο σε συνθήκες κανονικής χρήσης όσο και σε ακραίες περιπτώσεις (edge cases) όπως ταυτόχρονη πρόσβαση και λήξη συνεδριών.

5.4 Εκτέλεση Tests

Η εκτέλεση των tests πραγματοποιείται μέσω του εργαλείου Maven [10], με την εντολή:

mvn test

Όλα τα 49 tests (24 integration + 25 unit) εκτελούνται επιτυχώς, επιβεβαιώνοντας τη σωστή λειτουργία του REST API, της διαχείρισης συνεδριών, και της διεπαφής χρήστη σε όλα τα επίπεδα του συστήματος.

Κεφάλαιο 6. Σύνοψη

6.1 Επισκόπηση

Στην παρούσα διπλωματική εργασία υλοποιήθηκε η μετατροπή της desktop εφαρμογής Plutarch's Parallel Lives σε web εφαρμογή, με στόχο τη βελτίωση της προσβασιμότητας, της επεκτασιμότητας και της δυνατότητας υποστήριξης πολλαπλών χρηστών. Η μετάβαση πραγματοποιήθηκε με ελάχιστες τροποποιήσεις στον υπάρχοντα backend κώδικα, αξιοποιώντας τη σαφώς ορισμένη αρχιτεκτονική της αρχικής εφαρμογής και επεκτείνοντάς την όπου απαιτήθηκε για τη νέα λειτουργικότητα.

Η web έκδοση επιτρέπει πλέον τη χρήση της εφαρμογής μέσω φυλλομετρητή (browser), χωρίς ανάγκη τοπικής εγκατάστασης, διατηρώντας παράλληλα τη βασική φιλοσοφία, τις δυνατότητες ανάλυσης και τα εργαλεία οπτικοποίησης της desktop έκδοσης.

6.2 Συνεισφορά της Εργασίας

Οι βασικές συνεισφορές της παρούσας εργασίας συνοψίζονται στα εξής:

- Υλοποίηση REST API που εκθέτει τη λειτουργικότητα του backend μέσω 17 endpoints.
- Εισαγωγή μηχανισμού διαχείρισης συνεδριών (session management) για την υποστήριξη πολλαπλών ταυτόχρονων χρηστών.
- Ανάπτυξη σύγχρονου web frontend τύπου Single Page Application.
- Δημιουργία 49 αυτοματοποιημένων tests (24 integration tests για το REST API, 25 unit tests για τον ProjectStore).
- Διατήρηση του υπάρχοντος backend κώδικα με ελάχιστες στοχευμένες επεκτάσεις, εξασφαλίζοντας σταθερότητα και συμβατότητα με την desktop έκδοση.

6.3 Παραδοτέα

Τα βασικά παραδοτέα της εργασίας παρουσιάζονται στον παρακάτω πίνακα:

Παραδοτέο	**Περιγραφή**
DaintinessRestAPI.java	Υλοποίηση REST Controller (~925 γραμμές κώδικα)
ProjectStore.java	Διαχείριση συνεδριών χρηστών (~125 γραμμές κώδικα)
GlobalExceptionHandler.java	Κεντρικός χειρισμός σφαλμάτων (~85 γραμμές κώδικα)
index.html	Frontend Single Page Application (~2570 γραμμές κώδικα)
Tests	49 αυτοματοποιημένα tests (24 integration + 25 unit)

6.4 Μελλοντικές Επεκτάσεις

Η αρχιτεκτονική της εφαρμογής επιτρέπει περαιτέρω επεκτάσεις, τόσο σε επίπεδο λειτουργικότητας όσο και σε τεχνικό επίπεδο.

6.4.1 Λειτουργικές Επεκτάσεις

Φιλτράρισμα βάσει Μοτίβων (Pattern Filtering): Η προσθήκη δυνατότητας φιλτραρίσματος και επισήμανσης κελιών βάσει των ανιχνευμένων μοτίβων θα διευκόλυνε σημαντικά την ανάλυση μεγάλων συνόλων δεδομένων. Ο χρήστης θα μπορούσε να επιλέξει ένα ή περισσότερα μοτίβα (π.χ. Multiple Births, Multiple Deaths, Ladder) και η εφαρμογή να εμφανίζει αποκλειστικά ή να επισημαίνει οπτικά μόνο τα κελιά που τα περιέχουν, επιτρέποντας την ταχεία εστίαση σε περιοχές ενδιαφέροντος χωρίς οπτικό θόρυβο.

Σχολιασμοί και Σελιδοδείκτες (Annotations & Bookmarks): Η δυνατότητα προσθήκης σχολίων και σελιδοδεικτών σε συγκεκριμένα κελιά ή ομάδες κελιών θα μετέτρεπε την εφαρμογή σε πληρέστερο εργαλείο ανάλυσης. Ο χρήστης θα μπορούσε να σημειώνει ευρήματα, υποθέσεις ή παρατηρήσεις κατά τη διάρκεια της εξερεύνησης

των δεδομένων, με τις σημειώσεις να αποθηκεύονται ως μέρος του exported project και να είναι διαθέσιμες σε μελλοντικές συνεδρίες ή για διαμοιρασμό με συνεργάτες.

Συγκριτική Ανάλυση Πολλαπλών Datasets (Comparative Analysis): Μια σημαντική επέκταση θα ήταν η δυνατότητα φόρτωσης και ταυτόχρονης οπτικοποίησης δύο ή περισσότερων συνόλων δεδομένων για συγκριτική ανάλυση. Αυτό θα επέτρεπε, για παράδειγμα, τη σύγκριση της εξέλιξης δύο διαφορετικών βάσεων δεδομένων ή την παρακολούθηση του ίδιου συστήματος σε διαφορετικές χρονικές περιόδους, με παράλληλη ή επικαλυπτόμενη απεικόνιση και αυτόματη επισήμανση των διαφορών.

6.4.2 Τεχνικές Επεκτάσεις

Persistent storage: Αποθήκευση δεδομένων και projects σε βάση δεδομένων για μόνιμη διατήρηση.

Authentication & Authorization: Εισαγωγή συστήματος αυθεντικοποίησης χρηστών (login) και διαχείρισης δικαιωμάτων.

6.5 Συμπέρασμα

Η παρούσα διπλωματική εργασία πέτυχε τον στόχο της μετάβασης της εφαρμογής Plutarch's Parallel Lives σε web περιβάλλον, προσφέροντας μια σύγχρονη, προσβάσιμη και επεκτάσιμη λύση. Η εφαρμογή είναι πλέον διαθέσιμη μέσω browser, χωρίς την ανάγκη εγκατάστασης, ενώ ταυτόχρονα διατηρεί τον επιστημονικό και αναλυτικό χαρακτήρα της αρχικής υλοποίησης.

Η προσέγγιση που ακολουθήθηκε αποδεικνύει ότι η ύπαρξη καθαρού διαχωρισμού επιπέδων και γενικευμένου μοντέλου δεδομένων διευκολύνει σημαντικά την προσαρμογή υφιστάμενων εφαρμογών σε σύγχρονες τεχνολογικές πλατφόρμες.

Κεφάλαιο 7. Βιβλιογραφία

[1] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[2] Spring Framework, "Spring Boot Reference Documentation," https://docs.spring.io/spring-boot/docs/current/reference/html/

[3] JUnit Team, "JUnit 5 User Guide," https://junit.org/junit5/docs/current/user-guide/

[4] Mozilla Developer Network, "JavaScript Guide," https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[5] Chart.js, "Chart.js Documentation," https://www.chartjs.org/docs/latest/

[6] Φ. Γιάχος, Π. Βασιλειάδης, Α. Ζάρρας, Χ. Μπατσίλας, Ν. Παντελίδης "Ανάπτυξη Εργαλείου για την Ανάλυση της Εξέλιξης Βάσεων Δεδομένων," Διπλωματική εργασία, Πανεπιστήμιο Ιωαννίνων, 2015 https://www.cs.uoi.gr/~pvassil/supervision/diplomatikes/extraMaterial/PloutarchParallelLives/2023_PPL.pdf

[7] Γ. Παντελίδης, "Επέκταση Εργαλείου Ανάλυσης Εξέλιξης Βάσεων Δεδομένων με Υποστήριξη Γενικευμένων Δεδομένων," Διπλωματική εργασία, Πανεπιστήμιο Ιωαννίνων, 2021 https://www.cs.uoi.gr/~pvassil/supervision/diplomatikes/extraMaterial/PloutarchParallelLives/Pantelidis_DiplomaThesis_Final.pdf

[8] Κ. Μπατσίλας, "Αναβάθμιση Desktop Εφαρμογής Plutarch Parallel Lives με Νέες Λειτουργίες Οπτικοποίησης," Διπλωματική εργασία, Πανεπιστήμιο Ιωαννίνων, 2021 https://www.cs.uoi.gr/~pvassil/supervision/diplomatikes/extraMaterial/PloutarchParallelLives/Batsilas_Diploma.pdf

[9] Oracle Corporation, "Java SE Documentation," https://docs.oracle.com/en/java/

[10] Apache Maven Project, "Maven – Welcome to Apache Maven," https://maven.apache.org/

[11] W3C, "HTML5 Specification," https://www.w3.org/TR/html52/